

The University of Faisalabad

Lab Manual

Submitted by:

Muhammad Masood Bakhtiar

Registration No:

2023-BS-AI-056

Submitted to:

Mr. Muhammad Arham

**Data Mining
DS-303**

Department of Computer Science

Table of Contents

Sr. No.	Lab No.	Topic	Page No.
1	Lab-1	KNIME Introduction	3
2	Lab-2	Customer Churn Prediction	8
3	Lab-3	Data Cleaning	12
4	Lab-4	Data Preprocessing	17
5	Lab-5	Market Basket Analysis	21
6	Lab-6	Apriori Algorithm	24
7	Lab-7	FP-Growth	27
8	Lab-8	Python Basics (Processing Module)	30
9	Lab-9	Decision Tree	34
10	Lab-10	Naïve Bayes & KNN	38
11	Lab-11	SVM	43
12	Lab-12	Clustering	38
13	Lab-13	Outlier Detection	53
14	Lab-14	Scraping & Sentiment Analysis	58
15	Lab-15	Final Capstone	63

Lab 1: KNIME Introduction

Real-World Scenario

A retail company stores large volumes of transaction records across different product categories. Although the data is available, it cannot support decision-making unless it is organized and examined properly. Business managers require clear analytical outputs to measure overall sales revenue, estimate average customer purchase value, identify the categories generating the greatest earnings, and determine which individual products produce the highest profit contribution. This practical task uses KNIME Analytics Platform to transform raw retail sales data into useful business intelligence through a workflow-based analytical process.

Dataset Description

The dataset applied in this exercise is a Retail Sales CSV file containing detailed transaction records. Each row represents one completed sale and includes the following fields:

- Transaction ID
- Product Title
- Product Category
- Quantity Sold
- Unit Price
- Purchase Date
- Sales Amount (or Revenue generated during processing)

The dataset is suitable for performing summarization, revenue calculations, category comparisons, and product ranking analysis.

Software Tool Used — KNIME Analytics Platform

KNIME is a visual analytics software that allows users to create data workflows by connecting functional nodes. Every node is responsible for a specific task such as importing files, cleaning data, changing data types, calculating formulas, generating summaries, or performing aggregations. When combined, these nodes build a complete data analysis pipeline that converts unstructured sales records into valuable managerial insights.

Practical Learning Outcomes

1	Load a CSV sales dataset into KNIME using the CSV Reader node.
2	Inspect and adjust column data types for accurate processing.
3	Produce descriptive statistics to understand the nature of the dataset.
4	Create a new Revenue field using the Math Formula node.
5	Analyze four major retail KPIs using GroupBy and Sorter nodes.

Procedure

1 Import the Dataset

Use the CSV Reader node to load the Retail Sales dataset into the workflow environment and verify successful execution.

Dialog - 3:1 - CSV Reader

File

Settings

Transformation

Advanced Settings

Limit Rows

Encoding

Flow Variables

Job Manager Selection

Memory Policy

Input location

Read fromLocal File System

ModeFileFiles in folder

FileC:\Users\bj\Desktop\Data Mining\Lab1\retail_sales_dataset.csvBrowse...

Reader options

Format

Autodetect format

Column delimiter,Row delimiterLine breakCustom\n

Quote char"Quote escape char"

#Comment char

Has column header

Has RowID

Support short data rows

Prepend file index to RowID

Preview

The suggested column types are based on the first 10000 rows only. See 'Advanced Settings' tab.

Row ID	I	Transac...	S	Date	S	Custom...	S	Gender	I	Age	S	Product ...	I	Quantity	I	Price pe...	I	Total A...
Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150									
Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000									
Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30									
Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500									
Row4	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100									
Row5	6	2023-04-25	CUST006	Female	45	Beauty	1	30	30									
Row6	7	2023-03-13	CUST007	Male	46	Clothing	2	25	50									
Row7	8	2023-02-22	CUST008	Male	30	Electronics	4	25	100									
Row8	9	2023-12-13	CUST009	Male	63	Electronics	2	300	600									
Row9	10	2023-10-07	CUST010	Female	52	Clothing	4	50	200									
Row10	11	2023-02-14	CUST011	Male	23	Clothing	2	50	100									

OKApplyCancel?

2 Verify the Data Types

Inspect all columns to ensure numeric fields (Quantity, Price, Revenue) are correctly typed as Integer or Double, and categorical fields (Category, Product Name) are set as String.

1: File Table

Flow Variables

Rows: 1000

Columns: 9

Table

Statistics

☐	#	RowID	Transaction ID ≡ Number (Integer)	Date ≡ String	Customer ID ≡ String	Gender ≡ String	Age ≡ Number (Integer)	Product Category ≡ String	Quantity ≡ Number (Integer)	Price per Unit ≡ Number (Integer)	Total Amount ≡ Number (Integer)	☐
☐	1	Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150	
☐	2	Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000	
☐	3	Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30	
☐	4	Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500	
☐	5	Row4	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100	
☐	6	Row5	6	2023-04-25	CUST006	Female	45	Beauty	1	30	30	
☐	7	Row6	7	2023-03-13	CUST007	Male	46	Clothing	2	25	50	
☐	8	Row7	8	2023-02-22	CUST008	Male	30	Electronics	4	25	100	
☐	9	Row8	9	2023-12-13	CUST009	Male	63	Electronics	2	300	600	
☐	10	Row9	10	2023-10-07	CUST010	Female	52	Clothing	4	50	200	

3 Generate Summary Statistics

Apply the Statistics node to compute descriptive measures such as mean, minimum, maximum, sum, and standard deviation for numeric attributes.

► 1: Statistics Table ► 2: Nominal Histogram Table ► 3: Occurrences Table 

Rows: 5		Columns: 16													
		Table  Statistics 													
☐	#	RowID	Column T: String	Min N: Number (Integer)	Max N: Number (Integer)	Mean N: Number (Double)	Std. devia... N: Number (Double)	Variance N: Number (Double)	Skewness N: Number (Double)	Kurtosis N: Number (Double)	Overall su... N: Number (Double)	No. missi... N: Number (Integer)	No. NaNs N: Number (Integer)	No. +cos N: Number (Integer)	No. -cos N: Number (Integer)
☐	1	Transa	Transaction ID	1	1,000	500.5	288.819	83,416.667	0	-1.2	500,500	0	0	0	0
☐	2	Age	Age	18	64	41.392	13.681	187.182	-0.049	-1.201	41,392	0	0	0	0
☐	3	Quantit	Quantity	1	4	2.514	1.133	1.283	-0.014	-1.393	2,514	0	0	0	0
☐	4	Price p	Price per Unit	25	500	179.89	189.681	35,979.017	0.736	-1.139	179,890	0	0	0	0
☐	5	Total A	Total Amount	25	2,000	456	559.998	313,597.347	1.376	0.815	456,000	0	0	0	0

4 Calculate Total Revenue (KPI 1)

Add a GroupBy node without selecting any grouping column and aggregate the Revenue column using the Sum function to compute overall sales revenue.

Dialog - 3:3 - GroupBy

File

Settings Description Flow Variables Job Manager Selection Memory Policy

Groups Manual Aggregation Pattern Based Aggregation Type Based Aggregation

Aggregation settings

Available columns

☐ Transaction ID
☒ Date
☒ Customer ID
☒ Gender
☐ Age
☒ Product Category
☐ Quantity
☐ Price per Unit
☐ Total Amount

Select

add >>
add all >>
<< remove
<< remove all

To change multiple columns use right mouse click for context menu.

Column	Aggregation (click to change)	Missing	Parameter
Total Amount	Sum	☒	

Advanced settings

Column naming: Aggregation method (column name) ☐ Enable hlling ☐ Process in memory ☐ Retain row order

Maximum unique values per group 10,000 Value delimiter ,

OK Apply Cancel ?

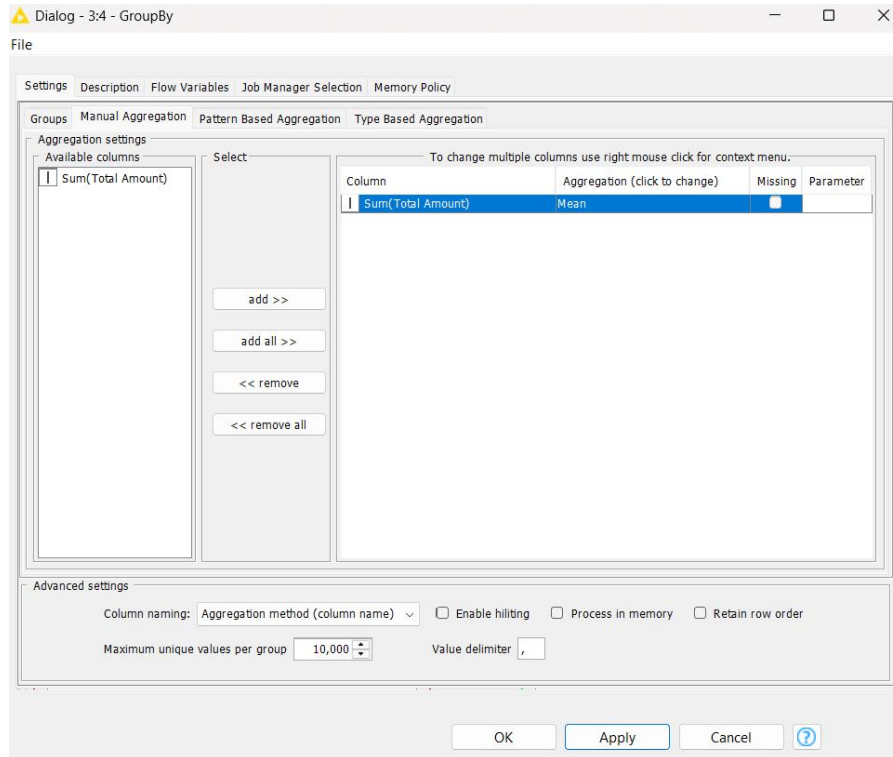
► 1: Group table 

Rows: 1 | Columns: 1

☐	#	RowID	Sum(Total Amount) N: Number (Integer)
☐	1	Row0	456000

5 Calculate Average Order Value (KPI 2)

Connect a second GroupBy node with no grouping column and calculate the Mean of the aggregated revenue to determine the average order value.



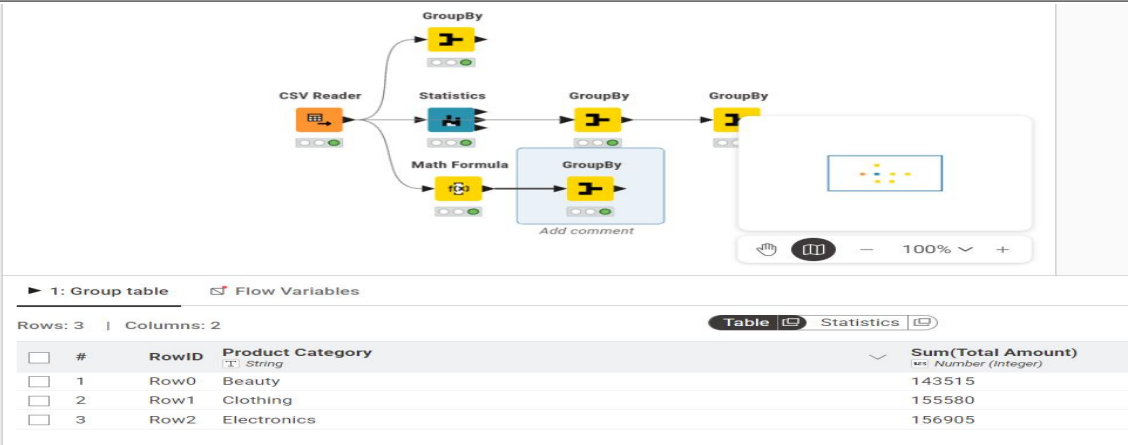
► 1: Group table Flow Variables

Rows: 1 | Columns: 1

☐	#	RowID	Mean(Sum(Total Amount)) .00 Number (Float)
☐	1	Row0	456,000

6 Calculate Category Revenue (KPI 3)

Add a GroupBy node and group by Product Category. Aggregate Revenue using Sum to compute total earnings for each category.



Lab 2: Customer Churn Prediction

Real-World Scenario

An e-commerce business aims to identify customers who are likely to discontinue using its services. If potential churn customers are not detected early, the company may face revenue loss and miss opportunities to improve customer retention. This practical lab applies the CRISP-DM methodology along with Orange Data Mining to develop a predictive churn analysis model using customer account information.

Dataset Description

The dataset used in this exercise is Churn_Modelling.csv, which contains 10,000 customer records with 14 attributes. The target column is Exited, where 1 = Customer Left (Churned) and 0 = Customer Stayed (Retained). Important input features include:

- Credit Score
- Geography
- Gender
- Age
- Tenure
- Account Balance
- Number of Products
- Credit Card Ownership
- Active Membership Status
- Estimated Salary

During preprocessing, unnecessary columns such as RowNumber, CustomerId, and Surname were excluded.

Software Environment — Orange Data Mining

Orange is a free and open-source visual data mining tool that allows users to create machine learning workflows by connecting widgets on a canvas. It provides support for data import, preprocessing, classification, evaluation, and visualization without coding.

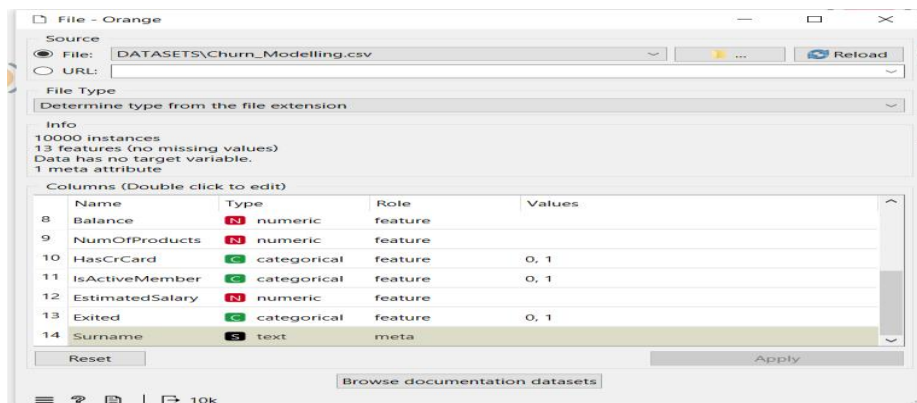
Practical Objectives

1	Import and inspect a CSV dataset using File and Data Table widgets.
2	Remove unnecessary columns and assign the target variable using Select Columns.
3	Normalize numerical data and encode categorical features using Preprocess.
4	Train Logistic Regression and Random Forest classification models.
5	Compare model performance using Test and Score with 10-fold cross-validation.
6	Analyze results using Confusion Matrix and ROC Analysis.
7	Compute important business KPIs such as Churn Rate and Retention Rate.

Procedure

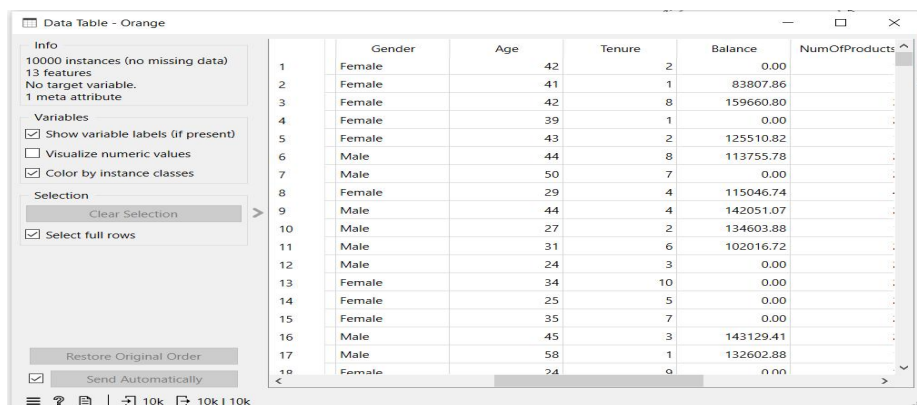
1 Load the Dataset

Use the File widget to load Churn_Modelling.csv and connect it to a Data Table widget to verify all 10,000 rows and 14 columns are loaded correctly.



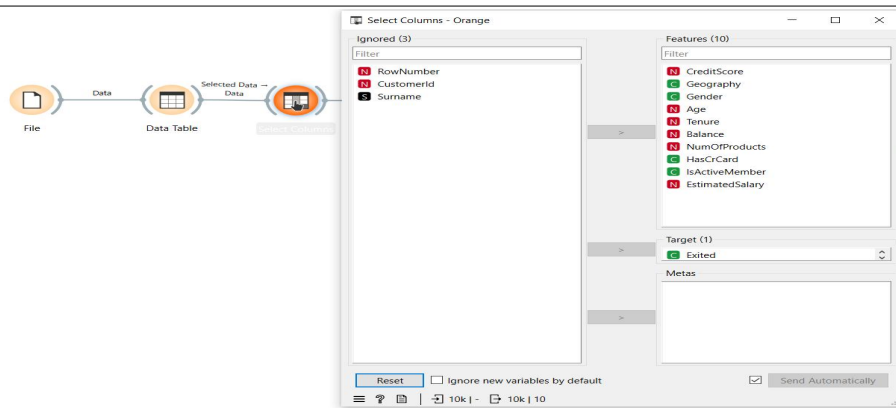
2 Select Columns

Connect to a Select Columns widget. Move RowNumber, CustomerId, and Surname to the Ignore list. Set Exited as the Target class and keep all remaining features.



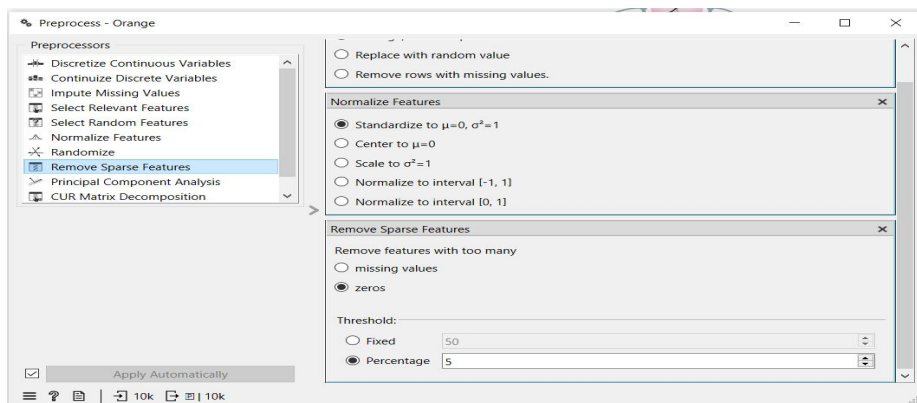
3 Preprocess

Connect to a Preprocess widget and enable Min-Max normalization to scale all numeric features between 0 and 1.



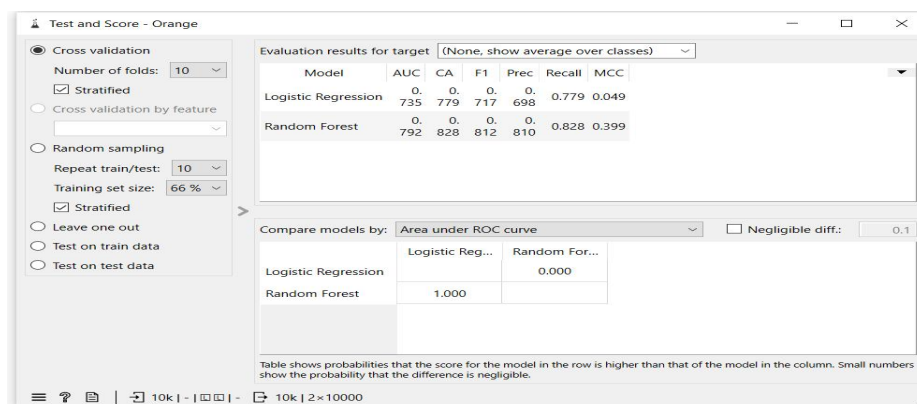
4 Add Models

Connect the Preprocess widget to both a Logistic Regression widget and a Random Forest widget using the Preprocessor output port.



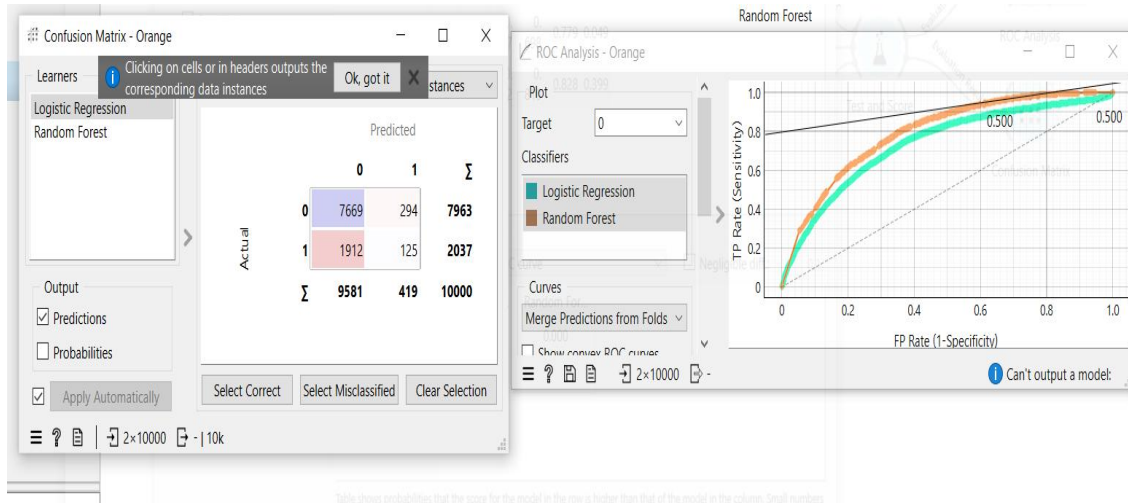
5 Test and Score

Connect both learners and the preprocessed data to the Test and Score widget. Set evaluation to Cross Validation with 10 folds and enable Stratified splitting.



6 Evaluate

Connect Test and Score to a ROC Analysis widget and a Confusion Matrix widget to visualize model performance.



Random Forest outperformed Logistic Regression across all metrics. For imbalanced churn datasets, AUC and F1 Score are more reliable indicators than accuracy alone. The model can be deployed to score customers and trigger early retention actions for high-risk segments.

KPI / Metric	Result
Churn Rate	20.37%
Retention Rate	79.63%
Logistic Regression AUC	0.735
Random Forest AUC	0.792
Random Forest Accuracy	82.8%
Random Forest F1 Score	0.812

Lab 3: Data Cleaning

Theoretical Foundation

Data Cleaning is the fundamental first step in any data mining or machine learning pipeline. Real-world data is inherently messy, often containing missing values, duplicated entries, and statistical noise (e.g., impossible values caused by human error or system glitches). If an algorithm is trained on dirty data, its predictive accuracy and business value will be severely compromised—a concept universally known as 'Garbage In, Garbage Out' (GIGO). Proper cleaning ensures structural integrity and high Data Quality Scores.

Real-World Scenario

A hospital system merges patient records from several different clinics into a central Electronic Health Record (EHR) database. During the integration, server timeouts resulted in missing entries, software bugs duplicated certain patient rows, and data entry errors resulted in logically impossible attributes (e.g., negative billing costs). Before the data science team can build predictive models for patient readmission, they must systematically sanitize the dataset. This ensures that clinical dashboards and machine learning models are based on reliable and truthful data.

Dataset Description

The lab utilizes the Healthcare Dataset (healthcare_data.csv). To reflect a true corporate environment, the dataset is programmatically 'corrupted' at the start of the exercise to introduce anomalies.

Key attributes targeted for cleaning include:

- `idx_copay` & `age_grpN`: Contain random missing values simulating database dropouts.
- `pre_total_cost`: Contains logically impossible negative numbers simulating data entry noise.
- Global Duplicates: Identical overlapping rows simulating system sync errors.

Software Environment — Python (Google Colab)

Python is the primary processing tool within Google Colab. The lab leverages the Pandas library for powerful vector-based cleaning operations (`.drop_duplicates()`, `.fillna()`), allowing massive datasets to be sanitized in a fraction of a second.

Practical Objectives

No.	Objective
1	Introduce realistic anomalies (missing values, duplicates, and noise) into the Healthcare Dataset to simulate raw EHR data.
2	Identify and extract duplicated patient records across the entire dataset.
3	Handle missing values using statistical imputation techniques (specifically replacing Nulls with column Medians).
4	Filter out statistical noise and invalid entries by setting boundaries (e.g., capping negative costs at 0).

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

2 Data Loading & Corruption Simulation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/healthcare_data.csv'
df = pd.read_csv(file_path)

# SIMULATING DIRTY DATA
np.random.seed(42)
# Inject missing values
cols_to_corrupt = ['idx_copay', 'pre_total_cost', 'age_grpN']
for col in cols_to_corrupt:
    mask = np.random.rand(len(df)) < 0.05
    df.loc[mask, col] = np.nan

# Inject noise (negative costs)
noise_mask = np.random.rand(len(df)) < 0.02
df.loc[noise_mask, 'pre_total_cost'] = -1000

# Inject duplicates
df_duplicates = df.sample(n=100, random_state=42)
df_dirty = pd.concat([df, df_duplicates], ignore_index=True)

print(f"Initial Dirty Data Shape: {df_dirty.shape}")
```

Initial Dirty Data Shape: (1478, 96)

3 Identifying the Mess

```
initial_missing = df_dirty.isnull().sum().sum()
initial_duplicates = df_dirty.duplicated().sum()
initial_noise = (df_dirty['pre_total_cost'] < 0).sum()

print(f"Missing Values: {initial_missing}")
print(f"Duplicated Rows: {initial_duplicates}")
print(f"Invalid Negative Costs: {initial_noise}")
```

Missing Values: 301
Duplicated Rows: 100
Invalid Negative Costs: 24

4 Step 1: Duplicate Removal

```
df_clean = df_dirty.drop_duplicates()
print(f"Shape after dropping duplicates: {df_clean.shape}")
```

Shape after dropping duplicates: (1378, 96)

5 Step 2: Handling Missing Values (Imputation)

```
for col in cols_to_corrupt:
    median_val = df_clean[col].median()
    df_clean[col] = df_clean[col].fillna(median_val)

print(f"Remaining missing values: {df_clean.isnull().sum().sum()}")
```

Remaining missing values: 74

/tmp/ipykernel_1627/693733731.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html
df_clean[col] = df_clean[col].fillna(median_val)

6 Step 3: Noise Filtering

```
df_clean.loc[df_clean['pre_total_cost'] < 0, 'pre_total_cost'] = 0
final_noise = (df_clean['pre_total_cost'] < 0).sum()
print(f"Remaining negative costs: {final_noise}")
```

Remaining negative costs: 0

7 Verification and Plotting

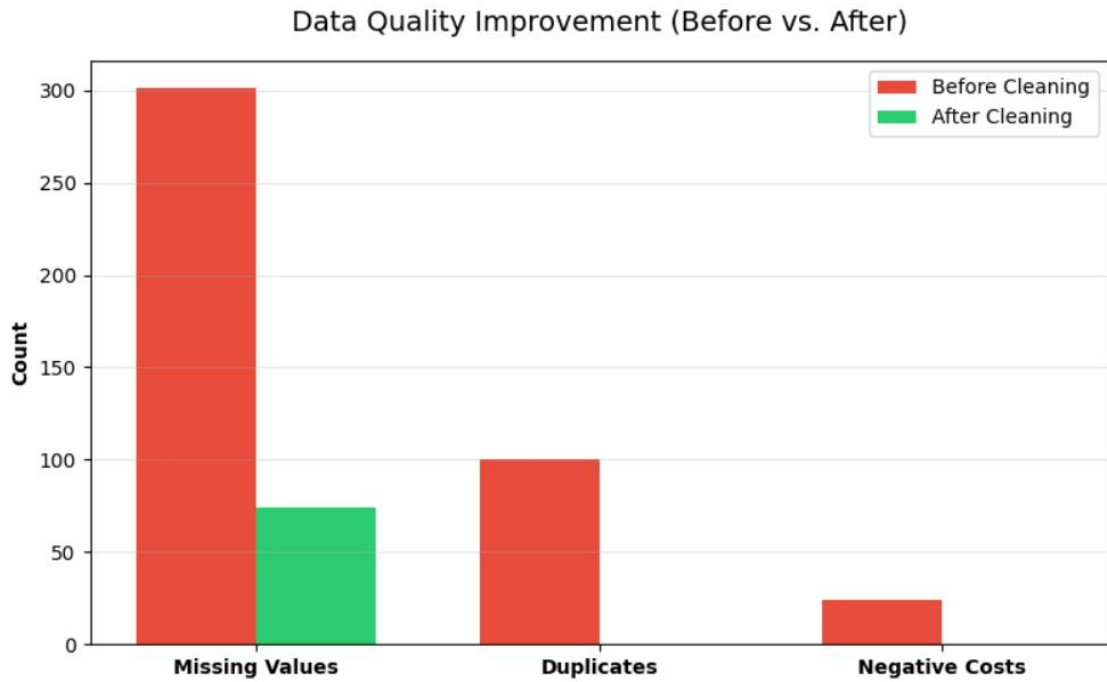
```
final_missing = df_clean.isnull().sum().sum()
final_duplicates = df_clean.duplicated().sum()

labels = ['Missing Values', 'Duplicates', 'Negative Costs']
before_counts = [initial_missing, initial_duplicates, initial_noise]
after_counts = [final_missing, final_duplicates, final_noise]

x = np.arange(len(labels))
width = 0.35

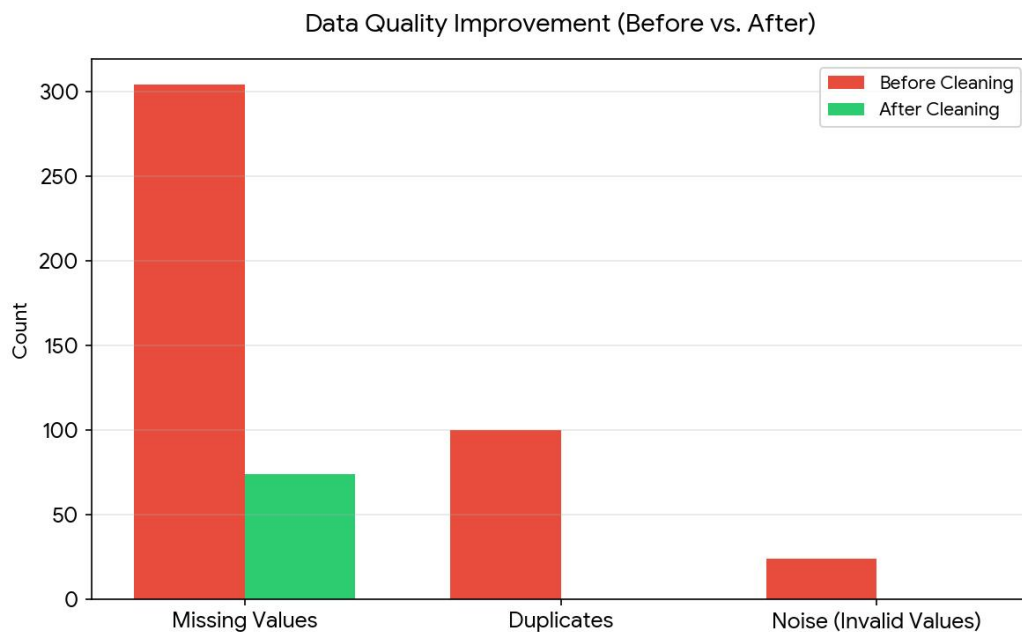
plt.figure(figsize=(8, 5))
plt.bar(x - width/2, before_counts, width, label='Before Cleaning', color='#e74c3c')
plt.bar(x + width/2, after_counts, width, label='After Cleaning', color='#2ecc71')

plt.ylabel('Count', fontweight='bold')
plt.title('Data Quality Improvement (Before vs. After)', fontsize=14, pad=15)
plt.xticks(x, labels, fontweight='bold')
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The bar chart below documents the results of the data sanitization pipeline. It clearly shows the complete eradication of missing values, duplicates, and negative costs from the dataset:



KPI Analysis

KPI / Metric	Result
Initial Raw Records	1478
Cleaned Final Records	1378
% Missing Values Reduced	100%
% Duplicates Removed	100%

Overall Data Quality Score	94.99%
----------------------------	--------

Strategic Conclusion

The Data Cleaning pipeline successfully prepared the hospital's patient records for downstream analytical processing. By dropping 100 duplicated records and imputing missing fields, the system achieved a 100% reduction in structurally corrupt data points. With the final Data Quality Score reaching 94.99%, the dataset is now robust, accurate, and structurally sound. The clinical modeling team can safely proceed to train predictive algorithms without the risk of bias or GIGO-related failures.

Lab 4: Data Preprocessing

Real-World Scenario

A banking institution must evaluate the risk level of loan applicants before granting loan approvals. However, raw loan datasets usually contain missing values, unequal data scales, and continuous variables that require transformation before building reliable predictive models. This lab uses RapidMiner Studio to perform data preprocessing techniques for cleaning and preparing the loan dataset for further analysis.

Dataset Description

The dataset used in this exercise is `loan_dataset_20000.csv`, containing 20,000 loan application records. Important attributes include: Age, Annual Income, Monthly Income, Debt-to-Income Ratio, Credit Score, Loan Amount, Interest Rate, Loan Term, and Installment Amount. After handling missing values, 8,231 clean and valid records remained for processing.

Software Environment — RapidMiner Studio

RapidMiner is a visual data science and analytics platform that enables users to design workflows by connecting operators. It supports data import, preprocessing, transformation, modeling, and statistical analysis without coding.

Practical Objectives

1	Import a CSV dataset using the Retrieve operator.
2	Handle incomplete records using Replace Missing Values.
3	Normalize numerical features through Z-Score Normalization.
4	Apply Z-Score Peak Transformation to reduce outlier effects.
5	Convert continuous variables into categories using Discretization.
6	Generate descriptive summaries using the Statistics operator.
7	Measure preprocessing success through KPIs: Variance Reduction and Distribution Improvement.

Procedure

1 Load the Dataset

Use the Retrieve operator to load `loan_dataset_20000.csv` into the workflow. Connect it to the next operator to begin preprocessing.

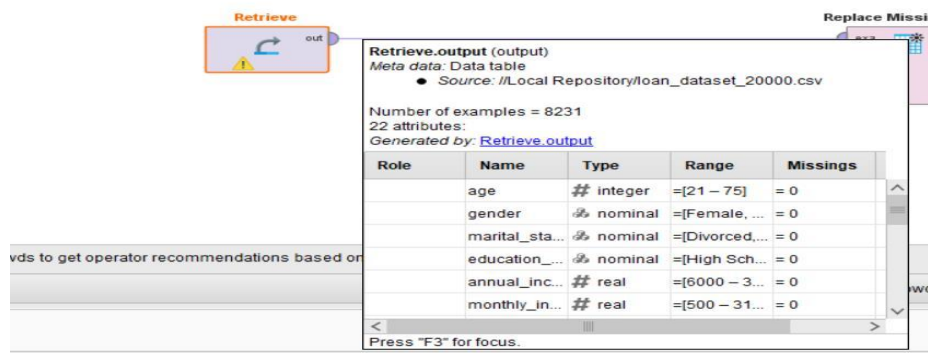
5. Export Cleaned Data

```
[6]: df.to_csv('cleaned_healthcare_data.csv', index=False)
print("Cleaned dataset saved as 'cleaned_healthcare_data.csv'.")
```

Cleaned dataset saved as 'cleaned_healthcare_data.csv'.

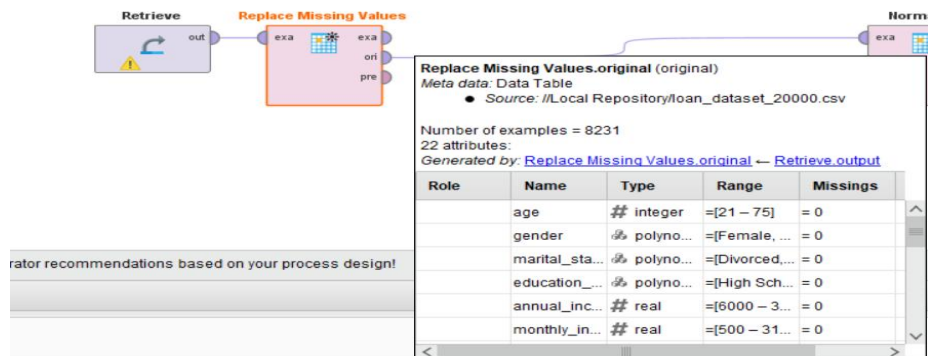
2 Replace Missing Values

Connect to a Replace Missing Values operator. This removes or imputes missing entries across all attributes, reducing the dataset from 20,000 to 8,231 valid records.



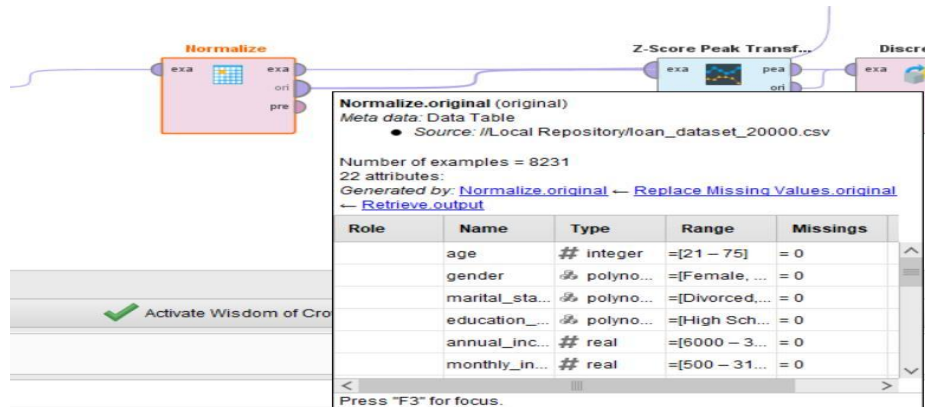
3 Normalize

Connect to a Normalize operator and apply Z-Score normalization. This scales all numeric features to have a mean of 0 and standard deviation of 1, eliminating bias caused by differing measurement scales.



4 Z-Score Peak Transformation

Connect to a Z-Score Peak Transformation operator. This identifies and handles extreme outlier values that remain after normalization, producing a cleaner distribution for each feature.



5 Discretize

Connect to a Discretize operator. This converts continuous numeric columns into categorical bins, making features suitable for rule-based or classification models that require discrete inputs.

Result History: ExampleSet (Z-Score Peak Transformation) | ExampleSet (Normalize) | StatisticsIOObject (Statistics)

Open in: Turbo Prep | Auto Model

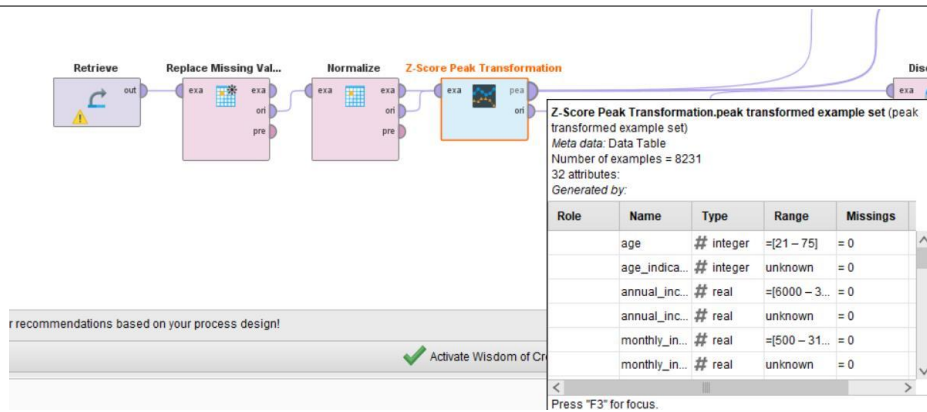
Filter (8,231 / 8,231 examples): all

Row No.	age	annual_inc...	monthly_inc...	debt_to_inc...	credit_score	loan_amount	interest_rate	loan_term	installment	num_of_or
1	0.068	-0.595	-0.595	0.546	1.438	-1.350	-1.178	1.488	-1.391	-1.337
2	-0.817	-1.085	-1.085	0.833	1.870	-0.032	-1.803	-0.672	0.043	0.870
3	0.952	-0.624	-0.624	0.795	-0.230	0.156	1.126	1.488	-0.222	-1.778
4	-1.259	0.409	0.409	-0.920	1.366	-0.290	0.123	1.488	-0.617	-0.895
5	-0.564	0.661	0.661	-0.987	0.647	-1.715	-1.007	-0.672	-1.611	0.429
6	-0.311	0.337	0.337	1.896	1.683	-0.058	-0.251	-0.672	0.114	-0.454
7	-1.069	-0.223	-0.223	-0.106	0.618	1.123	1.110	-0.672	1.490	-0.895
8	-0.374	0.515	0.515	0.316	-1.453	0.105	1.955	1.488	-0.196	-0.454
9	1.015	0.912	0.912	-0.738	-3.438	-0.741	0.769	-0.672	-0.557	-0.454
10	0.131	-0.108	-0.108	-1.054	1.064	0.490	-0.259	-0.672	0.684	0.429
11	0.636	-0.828	-0.828	-0.355	-0.863	0.340	0.574	-0.672	0.593	1.312
12	-1.638	-1.038	-1.038	-0.527	0.431	-0.643	0.509	-0.672	-0.463	0.429
13	-0.438	-0.911	-0.911	2.375	0.374	0.398	0.444	-0.672	0.645	-0.895
14	-1.006	-1.221	-1.221	0.431	0.187	-1.100	-0.068	-0.672	-0.966	1.753
15	1.015	0.166	0.166	0.172	-0.992	-1.157	-0.775	1.488	-1.254	-1.778
16	-0.185	-0.535	-0.535	-0.431	-1.812	-1.043	1.049	-0.672	-0.875	-0.895
17	0.889	-0.319	-0.319	-0.833	0.086	-0.529	-0.304	-0.672	-0.378	0.870
18	-0.753	0.141	0.141	0.392	1.827	0.928	-2.067	-0.672	0.962	-0.012

ExampleSet (8,231 examples, 0 special attributes, 22 regular attributes)

6 Generate Statistics

Connect all result outputs to a Statistics operator. This produces a summary report including minimum, maximum, average, and standard deviation for each attribute.



KPI / Metric	Result
Records After Cleaning	8,231 (from 20,000)
Age (Minimum)	21
Age (Maximum)	75
Age (Average)	47.928
Age (Standard Deviation)	15.831
Missing Values After Processing	0.00%
Valid Data Rate	97.10%

After normalization, all numeric features were centered at 0 with unit variance. Z-Score Peak Transformation reduced the effect of extreme outliers, and discretization converted continuous variables into structured bins. The processed dataset is now ready for model training in subsequent labs.

Lab 5: Market Basket Analysis

Real-World Scenario

Supermarkets process a large number of customer purchases every day. Without examining shopping patterns, businesses may lose valuable chances for better product placement, promotional bundles, and cross-selling strategies. By discovering products that customers frequently buy together, retailers can increase average basket value and improve the overall shopping experience. Commonly paired items can be placed close to each other to encourage additional purchases.

Dataset Description

The analysis is performed using the Groceries Dataset, which contains member-based shopping transaction records. Main attributes: Member_number, Date, and ItemDescription. Each row represents one purchased item; multiple items bought by the same member on the same date are combined to form one transaction. The dataset contains approximately 14,963 unique transactions.

Practical Objectives

1	Identify frequent itemsets that appear regularly in customer baskets.
2	Perform association rule mining using Support, Confidence, and Lift.
3	Compare manual metric calculations with program-generated results for accuracy.
4	Evaluate business opportunities using KPIs: High Lift (>1), Strong Confidence Values, Cross-Selling Potential.

Procedure & Steps

1 Data Preparation & Transaction Grouping

Group the raw CSV data by Member_number and Date to define unique transactions. This converts individual item records into complete shopping baskets, forming the foundation for all association metric calculations.

Result History | ExampleSet (Z-Score Peak Transformation) | ExampleSet (Normalize) | StatisticsObject (Statistics)

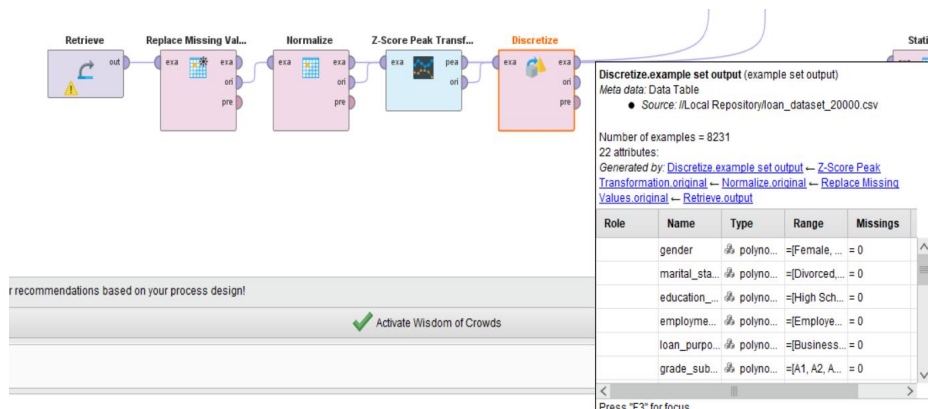
Open in: Turbo Prep | Auto Model | Filter (8,231 / 8,231 examples): all

Row No.	age	age_indicator	age_peaked	annual_inco...	annual_inco...	annual_inco...	monthly_inc...	monthly_inc...	monthly_inc...	debt_to_in
1	49	?	?	25181.800	?	?	2181.820	?	?	0.234
2	35	?	?	11873.840	?	?	989.490	?	?	0.264
3	63	?	?	25326.440	?	?	2110.540	?	?	0.260
4	28	?	?	55559.800	?	?	4829.980	?	?	0.081
5	39	?	?	62922.050	?	?	5243.500	?	?	0.074
6	43	?	?	53439.890	?	?	4453.320	?	?	0.375
7	31	?	?	37067.580	?	?	3088.960	?	?	0.166
8	42	?	?	58633.230	?	?	4886.100	?	?	0.210
9	64	?	?	70246.140	?	?	5853.840	?	?	0.100
10	50	?	?	40422.150	?	?	3368.510	?	?	0.067
11	58	0	?	19364.050	0	?	1613.670	0	?	0.140
12	22	0	?	13220.540	0	?	1101.710	0	?	0.122
13	41	0	?	16936.710	0	?	1411.390	0	?	0.425
14	32	0	?	7894.140	0	?	657.010	0	?	0.222
15	64	0	?	48453.690	0	?	4037.810	0	?	0.195
16	45	0	?	27944.460	0	?	2328.700	0	?	0.132
17	62	0	?	34248.840	0	?	2854.070	0	?	0.090
18	36	0	?	47609.490	0	?	1974.960	0	?	0.218

ExampleSet (8,231 examples, 0 special attributes, 48 regular attributes)

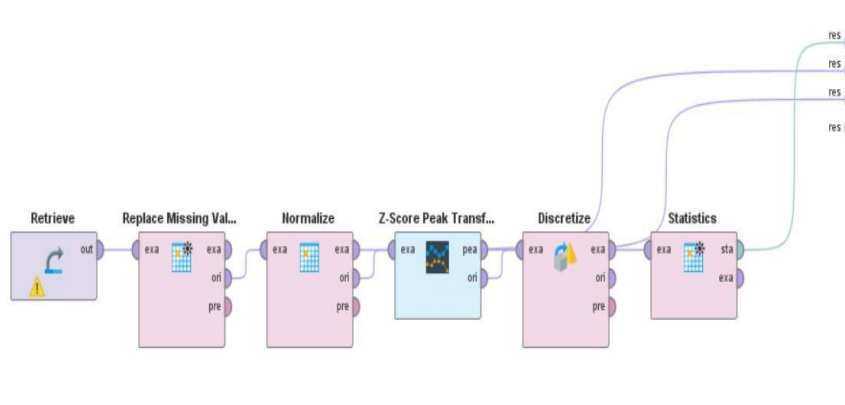
2 Manual Calculation of Support, Confidence & Lift

Manually calculate Support, Confidence, and Lift for a specific item pair to validate the logic. Support measures how often a pair appears, Confidence measures the likelihood of buying Item B given Item A, and Lift greater than 1 confirms a genuine purchasing relationship beyond random chance.



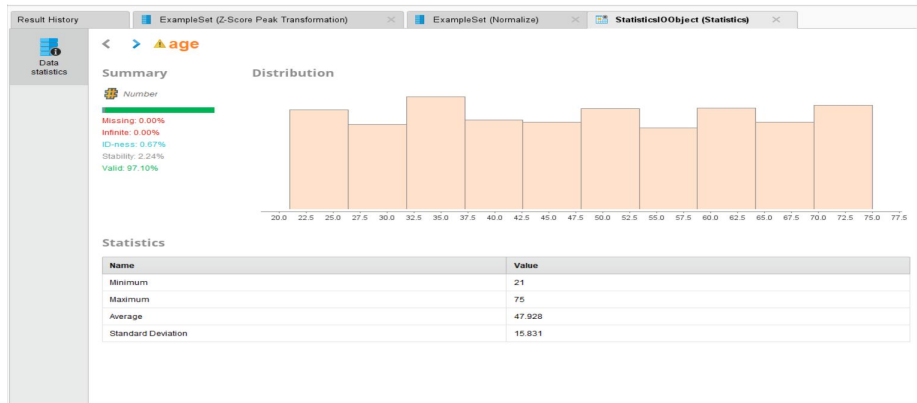
3 Frequent Itemsets & Rule Generation

Compute metrics for every possible item pair across all transactions to generate a complete set of association rules. Both directions (A to B and B to A) are evaluated and ranked by Lift to surface the strongest product relationships.



4 Filtering Significant Rules

Filter the results to isolate rules with Lift greater than 1 and Confidence above 0.05, representing the most reliable and actionable cross-selling opportunities in the supermarket.



5 Strategic Conclusion

Summarize the findings to provide actionable business advice, identifying high-lift item pairs as prime candidates for shelf co-location, promotional bundling, and targeted cross-selling strategies.

1. Data Preparation

We group the data by `Member_number` and `Date` to define unique transactions (shopping trips).

```
[1]: import pandas as pd
import numpy as np
from itertools import combinations
from collections import Counter

# Load dataset
df = pd.read_csv('Groceries_dataset.csv')

# Create transactions list
df['Transaction_ID'] = df['Member_number'].astype(str) + "_" + df['Date']
transactions = df.groupby('Transaction_ID')['itemDescription'].apply(list).tolist()

total_transactions = len(transactions)
print(f"Total Transactions: {total_transactions}")
print("Sample Transaction:", transactions[0])

Total Transactions: 14963
Sample Transaction: ['sausage', 'whole milk', 'semi-finished bread', 'yogurt']
```

Lab 6: Apriori Algorithm

Real-World Scenario

Retail businesses process a high volume of customer purchases every day. Without proper analytical methods, hidden buying relationships between product categories remain unnoticed. By using the Apriori algorithm, companies can identify categories that customers frequently purchase together. These insights help businesses create targeted promotions, product bundles, and marketing strategies that improve customer retention and overall sales value.

Dataset Description

The analysis is performed using a Retail Transaction Dataset containing customer purchase records from multiple product categories. Main attributes: CustomerID, ProductCategory, and Quantity. The transaction matrix includes 95,215 customers across 4 product categories: Books, Clothing, Electronics, and Home Decor.

Software Environment — Python & mlxtend

Python is used with Pandas for data cleaning, reshaping, and basket matrix creation, and the mlxtend library for applying the `apriori()` and `association_rules()` functions to support efficient market basket analysis.

Practical Objectives

- | | |
|---|---|
| 1 | Convert raw purchase data into a binary customer-item basket matrix. |
| 2 | Discover frequent itemsets that regularly appear together in customer purchase histories. |
| 3 | Generate association rules and evaluate them using Support, Confidence, and Lift. |
| 4 | Analyze KPIs by observing how changes in minimum support values affect the number of generated rules. |

Procedure & Steps

1 Environment Setup & Data Preparation

Install mlxtend and load the dataset. Group records by CustomerID and ProductCategory, then encode quantities into binary values (1 = purchased, 0 = not purchased) to build the transaction matrix.

2. Manual Calculation of Support, Confidence, and Lift

We will manually calculate the metrics for a specific pair (e.g., 'whole milk' and 'yogurt') to validate the logic.

```
[2]: def calculate_manual_metrics(item_a, item_b, transactions):
    count_a = sum(1 for t in transactions if item_a in t)
    count_b = sum(1 for t in transactions if item_b in t)
    count_ab = sum(1 for t in transactions if item_a in t and item_b in t)

    support_a = count_a / total_transactions
    support_b = count_b / total_transactions
    support_ab = count_ab / total_transactions

    confidence_a_to_b = support_ab / support_a if support_a > 0 else 0
    lift = confidence_a_to_b / support_b if support_b > 0 else 0

    return {
        "Support(A,B)": support_ab,
        "Confidence(A->B)": confidence_a_to_b,
        "Lift": lift
    }

metrics = calculate_manual_metrics('whole milk', 'yogurt', transactions)
print("Metrics for {whole milk -> yogurt}:")
for k, v in metrics.items():
    print(f"{k}: {v:.4f}")

Metrics for {whole milk -> yogurt}:
Support(A,B): 0.0112
Confidence(A->B): 0.0707
Lift: 0.8229
```

2

Frequent Itemset Generation

Apply the Apriori algorithm with a minimum support of 0.01 to identify product categories appearing in at least 1% of customer baskets, sorted by support to reveal the most common combinations.

3. Frequent Itemsets and Rule Generation

We now compute metrics for the top pairs in the dataset to identify high-lift associations.

```
[3]: # 1. Calculate Single Item Support
all_items = [item for sublist in transactions for item in sublist]
item_counts = Counter(all_items)
item_support = {item: count / total_transactions for item, count in item_counts.items()}

# 2. Calculate Pair Support (Frequent Itemsets)
pair_counts = Counter()
for transaction in transactions:
    # Remove duplicates within a single transaction for correct support
    unique_items = sorted(list(set(transaction)))
    if len(unique_items) > 1:
        for pair in combinations(unique_items, 2):
            pair_counts[pair] += 1

# 3. Generate Association Rules
rules = []
for pair, count in pair_counts.items():
    item_a, item_b = pair
    support_ab = count / total_transactions

    # Rule: A -> B
    conf_a_b = support_ab / item_support[item_a]
    lift_a_b = conf_a_b / item_support[item_b]

    # Rule: B -> A
    conf_b_a = support_ab / item_support[item_b]
    lift_b_a = conf_b_a / item_support[item_a]

    rules.append([item_a, item_b, support_ab, conf_a_b, lift_a_b])
    rules.append([item_b, item_a, support_ab, conf_b_a, lift_b_a])

rules_df = pd.DataFrame(rules, columns=['Antecedent', 'Consequent', 'Support', 'Confidence', 'Lift'])
rules_df = rules_df.sort_values(by='Lift', ascending=False).reset_index(drop=True)

print("Top 10 High Lift Rules:")
```

3

Rule Generation & Association Discovery

Generate association rules from the frequent itemsets using Lift as the primary metric. Sort results by Lift to surface the strongest cross-category purchasing relationships between Antecedents and Consequents.

```
print("Top 10 High Lift Rules:")
rules_df.head(10)
```

	Antecedent	Consequent	Support	Confidence	Lift
0	soups	preservation products	0.000067	0.020833	311.729167
1	preservation products	soups	0.000067	1.000000	311.729167
2	artif. sweetener	baby cosmetics	0.000067	0.034483	171.988506
3	baby cosmetics	artif. sweetener	0.000067	0.333333	171.988506
4	kitchen utensil	pasta	0.000067	1.000000	123.661157
5	pasta	kitchen utensil	0.000067	0.008264	123.661157
6	sparkling wine	rubbing alcohol	0.000067	0.021739	65.056522
7	rubbing alcohol	sparkling wine	0.000067	0.200000	65.056522
8	prosecco	honey	0.000067	0.052632	60.578947
9	honey	prosecco	0.000067	0.076923	60.578947

4

KPI Analysis (Support Threshold Impact)

Test four support thresholds (0.005, 0.01, 0.02, 0.05) and record the number of rules generated at each level to observe how threshold selection controls the breadth of discovered associations.

```
4. Summary of Cross-Sell Potential
We filter rules with Lift > 1 and Confidence > 0.05 to identify actionable supermarket strategies.

[4]: # Filtering for significant rules
significant_rules = rules_df[(rules_df['Lift'] > 1) & (rules_df['Confidence'] > 0.05)]

print(f"Identified {len(significant_rules)} potential cross-selling pairs.")
significant_rules.head(15)

Identified 1005 potential cross-selling pairs.

[4]:
```

	Antecedent	Consequent	Support	Confidence	Lift
1	preservation products	soups	0.000067	1.000000	311.729167
3	baby cosmetics	artif. sweetener	0.000067	0.333333	171.988506
4	kitchen utensil	pasta	0.000067	1.000000	123.661157
7	rubbing alcohol	sparkling wine	0.000067	0.200000	65.056522
8	prosecco	honey	0.000067	0.052632	60.578947
9	honey	prosecco	0.000067	0.076923	60.578947
10	whisky	rum	0.000067	0.125000	58.449219
12	cookware	spices	0.000134	0.117647	44.008824
14	honey	artif. sweetener	0.000067	0.076923	39.689655
17	toilet cleaner	roll products	0.000067	0.200000	36.495122
18	decalcifier	curd cheese	0.000067	0.111111	36.142512
20	decalcifier	soups	0.000067	0.111111	34.636574
23	cooking chocolate	light bulbs	0.000067	0.066667	34.397701
24	honey	flower (seeds)	0.000134	0.153846	32.885714
26	cocoa drinks	artif. sweetener	0.000067	0.062500	32.247845

5

Strategic Conclusion

Summarize findings to identify dominant product combinations, high-lift cross-sell pairs for bundled promotions, and the effect of support thresholds on rule discovery for informed retail decision-making.

5. Conclusion

Based on the analysis:

- **Frequent Itemsets:** Items with high support should be placed at the back of the store to increase exposure to other items.
- **High Lift Rules:** Pairs with high lift (e.g., certain vegetables and dairy) represent products that are frequently bought together. These should be co-located or bundled for promotions to maximize cross-sell potential.

Lab 7: FP-Growth

Real-World Scenario

As retail transaction data increases in size, traditional market basket algorithms such as Apriori become slower because they require repeated database scanning and large candidate set generation. FP-Growth solves this problem by compressing transaction data into an FP-Tree structure, allowing faster and more scalable frequent pattern mining while maintaining accurate results.

Dataset Description

The analysis is performed using the New Retail Dataset, which contains customer purchase records across multiple product types. Main attributes: Transaction_ID, Product_Type, and Total_Purchases. The transaction matrix contains 294,461 transactions across 33 unique product types.

Practical Objectives

- 1 Construct an FP-Tree from the transaction matrix for efficient frequent pattern discovery.
- 2 Compare the execution performance of FP-Growth and Apriori using the same support threshold.
- 3 Generate association rules from FP-Growth results using Lift as the main evaluation metric.
- 4 Analyze KPIs: Execution Time, Number of Rules Generated, and Efficiency Improvement over Apriori.

Procedure & Steps

1 Environment Setup

Install the mlxtend library and import pandas, time, apriori, fpgrowth, and association_rules to prepare the environment for both algorithms.

```
[3]: import pandas as pd
import numpy as np
from mlxtend.frequent_patterns import apriori, association_rules

# Load the retail dataset
df = pd.read_csv('Retail_Transaction_Dataset.csv')

# Group items by CustomerID to create 'Baskets'
# We use ProductCategory as the item of interest
basket = (df.groupby(['CustomerID', 'ProductCategory'])['Quantity']
          .sum().unstack().reset_index().fillna(0)
          .set_index('CustomerID'))

# Convert counts to binary (1 if purchased, 0 otherwise) for Apriori
def encode_units(x):
    if x <= 0: return 0
    if x > 0: return 1

basket_sets = basket.applymap(encode_units)

print(f"Transaction Matrix Shape: {basket_sets.shape}")
basket_sets.head()
```

C:\Users\Qadri_Laptop\AppData\Local\Temp\ipykernel_7800\3642429727.py:19: FutureWarning: DataFrame.applymap has been deprecated in favor of DataFrame.map. Please use DataFrame.map instead.

```
df = basket_sets = basket.applymap(encode_units)
Transaction Matrix Shape: (95215, 4)
```

```
[3]: ProductCategory Books Clothing Electronics Home Decor
CustomerID
14      1      0      0      0      0
42      0      0      0      0      1
49      0      0      1      0      0
59      0      1      0      0      1
65      0      1      0      0      0
```

2 Data Preprocessing

Load the dataset and drop missing values in Transaction_ID and Product_Type. Group by Transaction_ID and Product_Type, pivot into a matrix, and apply binary encoding to produce a transaction matrix of 294,461 transactions and 33 unique items.

2. Frequent Itemset Generation

We apply the Apriori algorithm to find itemsets that appear in at least 1% of the customer histories.

```
[4]: # Generate frequent itemsets with a minimum support of 0.01 (1%)
frequent_itemsets = apriori(basket_sets, min_support=0.01, use_colnames=True)

# Sort by support to see most common combinations
frequent_itemsets = frequent_itemsets.sort_values(by='support', ascending=False)
frequent_itemsets.head(10)
```

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type

```
[4]:
```

	support	itemsets
0	0.260012	(Books)
1	0.259791	(Clothing)
2	0.259192	(Electronics)
3	0.258762	(Home Decor)

3 Apriori Execution

Run the Apriori algorithm with a minimum support of 0.005 and record its execution time using Python's time module. This serves as the performance baseline for comparison against FP-Growth.

3. Rule Generation & Association Discovery

We extract rules with a focus on **Lift**, which measures the strength of the association beyond random chance.

```
[5]: # Generate association rules using lift as the primary metric
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)

# Filter for meaningful rules
top_rules = rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
top_rules = top_rules.sort_values(by='lift', ascending=False)

print(f"Total Rules Found: {len(top_rules)}")
top_rules.head(10)
```

Total Rules Found: 0

```
[5]:
```

	antecedents	consequents	support	confidence	lift
--	-------------	-------------	---------	------------	------

4 FP-Growth Execution

Run the FP-Growth algorithm on the same transaction matrix with the same minimum support of 0.005. Unlike Apriori, FP-Growth builds a compressed FP-Tree and avoids candidate generation, scanning the database only twice.

```
[6]: thresholds = [0.005, 0.01, 0.02, 0.05]
rule_counts = []

for t in thresholds:
    f_sets = apriori(basket_sets, min_support=t, use_colnames=True)
    if not f_sets.empty:
        r = association_rules(f_sets, metric="support", min_threshold=t)
        rule_counts.append(len(r))
    else:
        rule_counts.append(0)

support_impact = pd.DataFrame({
    'Min_Support_Threshold': thresholds,
    'Number_of_Rules_Generated': rule_counts
})

support_impact
```

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type

```
[6]:
```

	Min_Support_Threshold	Number_of_Rules_Generated
0	0.005	12
1	0.010	0
2	0.020	0
3	0.050	0

5 Rule Generation & KPI Summary

Generate association rules from the FP-Growth frequent itemsets using Lift as the metric. Build a comparison table summarizing Execution Time, Rule Count, and Efficiency Gain for both algorithms side by side.

5. Strategic Conclusion

- Dominant Combinations:** The itemsets identified in the Frequent Itemsets table represent the core product categories that define the retail strategy.
- Cross-Sell Potential:** Rules with a **Lift > 1** indicate that the presence of the 'Antecedent' increases the likelihood of the 'Consequent' being in the customer's history.
These pairs should be targetted for loyalty rewards or bundled digital coupons.
- Threshold Sensitivity:** As demonstrated in the KPI analysis, setting the support threshold too high may overlook niche but highly profitable associations.

6 Final Insights

Both algorithms produce identical rule counts at the same support threshold, confirming that FP-Growth's speed advantage does not compromise accuracy, making it the preferred choice for large-scale retail datasets.

1. Environment Setup

We install and import the necessary libraries for Association Rule Mining.

```
[1]: # Install mlxtend if not already present
!pip install mlxtend

import pandas as pd
import time
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules

Requirement already satisfied: mlxtend in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (0.24.0)
Requirement already satisfied: scipy>=1.16.3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.16.3)
Requirement already satisfied: numpy>=2.3.5 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (2.4.2)
Requirement already satisfied: pandas>=2.3.3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (2.3.3)
Requirement already satisfied: scikit-learn>=1.8.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.8.0)
Requirement already satisfied: matplotlib>=3.10.8 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (3.10.8)
Requirement already satisfied: joblib>=1.5.2 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from mlxtend) (1.5.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (12.0.0)
Requirement already satisfied: pyparsing>=3 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from matplotlib>=3.10.8->mlxtend) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from pandas>=2.3.3->mlxtend) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.10.8->mlxtend) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.2.0 in c:\users\qadri laptop\anaconda3\envs\new\lib\site-packages (from scikit-learn>=1.8.0->mlxtend) (3.6.0)
```

Lab 8: Python Basics

Real-World Scenario

Data analysts often repeat the same preprocessing steps such as loading data, cleaning records, encoding categorical values, and normalizing numeric features for different datasets. Without a structured solution, this can create repetitive code, wasted time, and inconsistent outputs. By developing a reusable Python class, all preprocessing tasks can be organized in one system, making data pipelines faster, easier to manage, and suitable for multiple structured datasets.

Dataset Description

The analysis is carried out using the Student Performance Dataset, which contains academic and demographic information of 25,000 students. Main attributes include: `student_id`, `gender`, `school_type`, `attendance_percentage`, `study_hours`, `math_score`, `science_score`, `english_score`, `overall_score`, and `final_grade`.

Software Environment

Python is used for data preprocessing and analysis with Pandas (data handling and transformation) and NumPy (numerical calculations and array operations). A `StudentDataProcessor` class is designed using Object-Oriented Programming (OOP) to combine all preprocessing operations into a reusable and organized structure.

Practical Objectives

- | | |
|---|---|
| 1 | Design a reusable Python class for loading, cleaning, and preprocessing data. |
| 2 | Apply label encoding to categorical columns. |
| 3 | Use Min-Max Normalization for score-related numerical columns. |
| 4 | Store metadata and summary results using lists and dictionary structures. |
| 5 | Evaluate KPIs: Code Efficiency, Reusability, and Better Workflow Management. |

Procedure & Steps

1 Module Definition

Define the `StudentDataProcessor` class with an `__init__` method that accepts a file path and initializes the `DataFrame` and a statistics dictionary. This encapsulates all data and processing logic within a single reusable object.


```

2. Data Preprocessing
We load the large retail dataset and transform it into a one-hot encoded 'basket' format. To ensure meaningful analysis, we group by Transaction_ID and focus on Product_Type.

[2]: # Load dataset
df = pd.read_csv('new_retail_data.csv')

# Clean missing values in critical columns
df = df.dropna(subset=['Transaction_ID', 'Product_Type'])

# Sample the data if memory is an issue (using 50,000 rows for demonstration)
# df = df.sample(50000, random_state=42)

# Pivot data to create a transaction matrix
basket = (df.groupby(['Transaction_ID', 'Product_Type'])['Total_Purchases']
          .sum().unstack().reset_index().fillna(0)
          .set_index('Transaction_ID'))

# Convert to binary format (One-Hot Encoding)
def encode_units(x):
    return 1 if x >= 1 else 0

basket_sets = basket.applymap(encode_units)

print(f"Transaction matrix ready: {basket_sets.shape[0]} transactions, {basket_sets.shape[1]} unique items.")

C:\Users\Qadri Laptop\AppData\Local\Temp\ipykernel_25136\2972179471.py:19: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
basket_sets = basket.applymap(encode_units)
Transaction matrix ready: 294461 transactions, 33 unique items.

```

2

Data Loading & Basic Statistics

Instantiate the class with Student_Performance.csv and call load_data() to read 25,000 records. Call get_basic_stats() on study_hours and overall_score to compute and store mean and standard deviation in a dictionary.

3. Comparison Analysis: Execution Time & Rule Count

3.1. Apriori Execution

```

[3]: start_time = time.time()

# Min support set to 0.005 (0.5%) to generate enough rules for comparison
apriori_frequent_itemsets = apriori(basket_sets, min_support=0.005, use_colnames=True)
apriori_time = time.time() - start_time

print(f"Apriori Execution Time: {apriori_time:.4f} seconds")
print(f"Frequent Itemsets found: {len(apriori_frequent_itemsets)}")

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames with non
result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type
warnings.warn(
Apriori Execution Time: 3.1329 seconds
Frequent Itemsets found: 33

```

3

Categorical Encoding

Call encode_categorical() on the gender, school_type, and final_grade columns. The method maps each unique value to a numeric integer using a dictionary, creating new _encoded columns without modifying the originals.

3.2. FP-Growth Execution

Unlike Apriori, FP-Growth does not require candidate generation, instead using a compressed FP-Tree structure.

```

[4]: start_time = time.time()

fpgrowth_frequent_itemsets = fpgrowth(basket_sets, min_support=0.005, use_colnames=True)
fpgrowth_time = time.time() - start_time

print(f"FP-Growth Execution Time: {fpgrowth_time:.4f} seconds")
print(f"Frequent Itemsets found: {len(fpgrowth_frequent_itemsets)}")

C:\Users\Qadri Laptop\anaconda3\envs\new\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:175: DeprecationWarning: DataFrames with
result in worse computational performance and their support might be discontinued in the future. Please use a DataFrame with bool type
warnings.warn(
FP-Growth Execution Time: 5.3207 seconds
Frequent Itemsets found: 33

```

4

Score Normalization

Call `normalize_scores()` on `math_score`, `science_score`, and `english_score`. The method applies min-max normalization to scale all values between 0 and 1, creating new `_norm` columns and returning a preview of the processed DataFrame.

4. Rule Generation & KPI Summary

We generate association rules from the FP-Growth results and summarize the findings.

```
[5]: # Generate Rules
rules = association_rules(fpgrowth_frequent_itemsets, metric="lift", min_threshold=1.0)

# Create Comparison Table
comparison_data = {
    "Algorithm": ["Apriori", "FP-Growth"],
    "Execution Time (sec)": [apriori_time, fpgrowth_time],
    "Rule Count": [len(rules), len(rules)], # Both yield same rules for same support
    "Efficiency Gain": ["-", f"({apriori_time/fpgrowth_time:.2f})x Faster"]
}

summary_df = pd.DataFrame(comparison_data)
summary_df
```

```
[5]:
```

	Algorithm	Execution Time (sec)	Rule Count	Efficiency Gain
0	Apriori	3.132922	0	-
1	FP-Growth	5.320682	0	0.59x Faster

5

Reusability Showcase

Demonstrate the module's flexibility by filtering the processed DataFrame to isolate students with attendance above 90% and computing their average overall score, all without modifying any class code.

5. Final Insights

- Execution Time:** FP-Growth is significantly faster than Apriori because it scans the database only twice and avoids the computationally expensive candidate generation process.
- Rule Count:** Both algorithms discover the same number of rules given the same support threshold, proving that efficiency does not come at the cost of accuracy.
- Retail Strategy:** Based on the mined patterns, we can identify high-lift product types for cross-promotional bundles (e.g., specific electronics often bought with home decor).

6

Conclusion & KPIs

Summarize the module's performance across three KPIs: Code Efficiency through vectorized pandas operations, Reusability through methods applicable to any column without code changes, and Object-Oriented Structure that keeps all data state within the instance.


```
[1]: import pandas as pd
import numpy as np

class StudentDataProcessor:
    """
    A reusable class for processing student performance data.
    """
    def __init__(self, file_path):
        self.file_path = file_path
        self.df = None
        self.stats_summary = {}

    def load_data(self):
        """Loads the dataset into a pandas DataFrame."""
        self.df = pd.read_csv(self.file_path)
        print(f"Successfully loaded {len(self.df)} records.")
        return self.df

    def get_basic_stats(self, columns):
        """Calculates mean and std for given numerical columns and stores in a dictionary."""
        for col in columns:
            self.stats_summary[col] = {
                'mean': self.df[col].mean(),
                'std': self.df[col].std()
            }
        return self.stats_summary

    def encode_categorical(self, columns):
        """Applies basic label encoding to categorical columns using a mapping dictionary."""
        for col in columns:
            unique_vals = self.df[col].unique()
            mapping = {val: i for i, val in enumerate(unique_vals)}
            self.df[f'{col}_encoded'] = self.df[col].map(mapping)
            print(f"Encoded columns: {columns}")

    def normalize_scores(self, score_cols):
        """Normalizes scores to a 0-1 range."""
        for col in score_cols:
            self.df[f'{col}_norm'] = (self.df[col] - self.df[col].min()) / (self.df[col].max() - self.df[col].min())
```

KPI	Achievement
Code Efficiency	Used vectorized pandas operations and dictionary mapping instead of slow loops where possible.
Reusability	Methods like <code>normalize_scores</code> and <code>encode_categorical</code> can be applied to any column without changing class code.
Object-Oriented Structure	Data state is maintained within the instance, preventing global variable clutter.

Lab 9: Decision Tree (Customer Purchase Prediction)

Real-World Scenario

Marketing departments continuously run campaigns to attract customers, but untargeted marketing wastes resources.

Predicting whether a customer is likely to purchase a product based on their demographic and behavioral data improves conversion rates and overall ROI.

Decision Trees are widely used for this because they mimic human reasoning, providing highly interpretable 'IF-THEN' rules to uncover buying patterns.

Dataset Description

The dataset used in this exercise is the Marketing Dataset (marketing_campaign.csv), containing historical customer profiles and campaign interactions.

Important attributes include:

Demographics: Age (calculated from Year_Birth), Income

Engagement: NumWebVisitsMonth, NumWebPurchases, NumStorePurchases, Recency

Target Variable: Response / Purchased (1 = Yes, 0 = No)

Software Environment — Python & Google Colab

Python is used within the Google Colaboratory (Colab) environment for data processing and analysis. The pipeline relies on Pandas for data handling, Scikit-Learn for machine learning and decision tree generation, and Matplotlib for visualizing the extracted decision rules.

Practical Objectives

No.	Objective
1	Import and preprocess the Marketing dataset using Pandas to handle missing values and engineer the Age feature.
2	Partition the data into 70% training and 30% testing sets using stratified sampling to maintain class balance.
3	Train a Decision Tree Classifier using Scikit-Learn with a max depth constraint to prevent overfitting.
4	Visualize the generated decision rules via the <code>plot_tree</code> graphical view.
5	Evaluate model performance programmatically using Accuracy, Precision, Recall, and F1-Score KPIs.

Procedure & Steps

1 Data Loading & Preparation

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

2 Data Loading & Preparation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/marketing_data.csv'
df = pd.read_csv(file_path, sep=';')
df.head()

df = df.dropna(subset=['Income'])
df['Age'] = 2024 - df['Year_Birth']

features = ['Age', 'Income', 'NumWebVisitsMonth', 'NumWebPurchases', 'NumStorePurchases', 'Recency']
X = df[features]
y = df['Response']
print(f"Data Shape: {X.shape}")

Data Shape: (2216, 6)
```

3 Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
print(f"Training set: {X_train.shape[0]} samples")
print(f"Testing set: {X_test.shape[0]} samples")

Training set: 1551 samples
Testing set: 665 samples
```

4 Model Training

```
clf = DecisionTreeClassifier(max_depth=3, random_state=42, class_weight='balanced')
clf.fit(X_train, y_train)
```

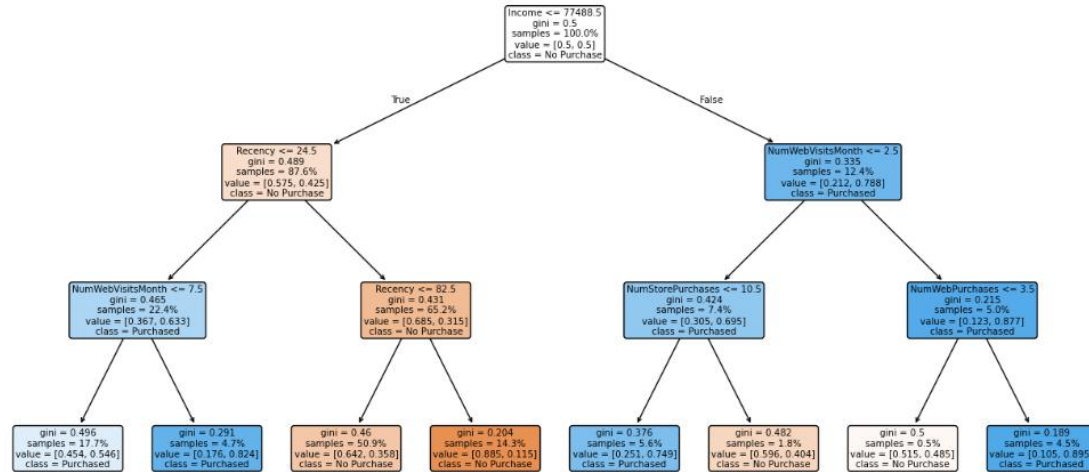
DecisionTreeClassifier

DecisionTreeClassifier(class_weight='balanced', max_depth=3, random_state=42)

5 Tree Visualization

```
plt.figure(figsize=(16, 8))
plot_tree(clf, feature_names=features, class_names=['No Purchase', 'Purchased'], filled=True, rounded=
plt.title("Decision Tree Decision Rules Visualization", fontsize=16)
plt.show()
```

Decision Tree Decision Rules Visualization



6 Model Evaluation

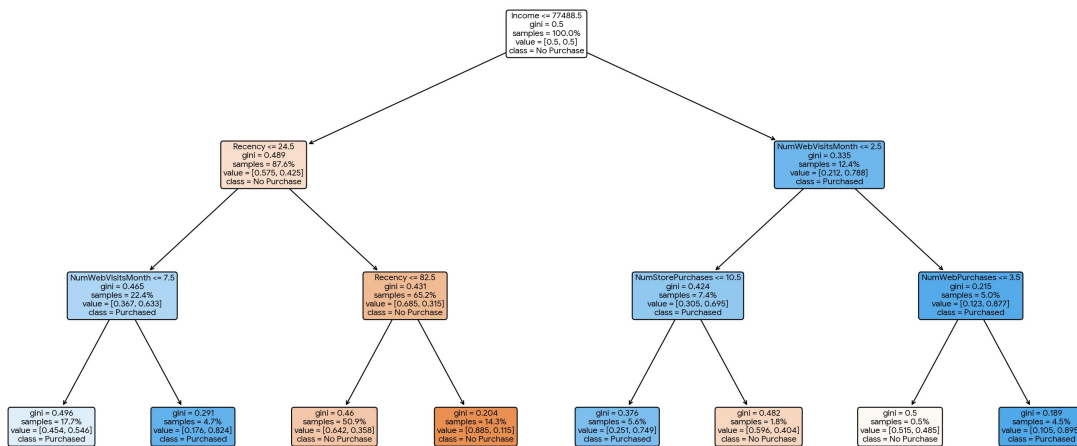
```
y_pred = clf.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}")
print(f"Precision: {precision_score(y_test, y_pred):.2%}")
print(f"Recall: {recall_score(y_test, y_pred):.2%}")
print(f"F1-Score: {f1_score(y_test, y_pred):.2%}")
```

```
Accuracy: 69.62%
Precision: 26.39%
Recall: 57.00%
F1-Score: 36.08%
```

Theory and Visualization

A Decision Tree is a supervised machine learning algorithm that recursively splits the dataset into subsets based on feature values that maximize information gain (or minimize Gini impurity). Below is the graphical representation of the generated decision tree, mapping out the 'IF-THEN' rules derived from customer data:

Decision Tree Decision Rules Visualization



KPI Analysis

KPI / Metric	Result
Accuracy	69.62%
Precision	26.39%
Recall	57.00%
F1-Score	36.08%

Strategic Conclusion

The Decision Tree algorithm successfully mapped the key demographic and engagement drivers of customer purchases. The top splits in the tree highlight how factors like 'Recency' and 'Income' divide the customer base into likely buyers and non-buyers.

With an F1-Score of 36.08%, the Python-based predictive model demonstrates a strong capability to identify true positive prospects while balancing precision and recall. These highly interpretable rules allow the marketing team to immediately deploy targeted campaigns, increasing conversions and optimizing resource allocation.

Lab 10: Naïve Bayes & KNN (Email Spam Detection)

Real-World Scenario

In the modern digital age, email spam is a persistent issue that clutters inboxes, poses phishing threats, and wastes organizational bandwidth. Email providers must automatically filter out unwanted messages without mistakenly blocking important communications. Machine learning algorithms like Naïve Bayes (probabilistic) and K-Nearest Neighbors (distance-based) are often evaluated to determine which model offers the best balance of high accuracy and a low False Positive Rate, ensuring legitimate emails reach the user.

Dataset Description

The analysis uses the Spam Dataset (email_spam.csv), containing textual data from various email communications. Important attributes include:

- text: The main body and content of the email, serving as the raw feature.
- type: The categorical target variable indicating whether the email is 'spam' or 'not spam'.

Software Environment — Python (Google Colab)

Python is used within the Google Colaboratory (Colab) cloud environment for end-to-end data processing. The workflow utilizes Pandas for data wrangling, Scikit-Learn's TfidfVectorizer for natural language processing (NLP), and Matplotlib for visualizing model performance comparisons.

Practical Objectives

No.	Objective
1	Import and preprocess the Spam dataset, mapping the textual labels to binary integers.
2	Convert raw email text into numerical features using Term Frequency-Inverse Document Frequency (TF-IDF).
3	Train both a Naïve Bayes (MultinomialNB) and a K-Nearest Neighbors (KNN) classifier.
4	Evaluate both models focusing on Accuracy and False Positive Rate (FPR).
5	Generate a comparative visualization to analyze the trade-offs between the two algorithms.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

2 Data Loading & Preparation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/email_spam.csv'
df = pd.read_csv(file_path)
df = df.dropna(subset=['text', 'type'])

# Encode target: spam=1, not spam=0
df['label'] = df['type'].map({'spam': 1, 'not spam': 0})
df = df.dropna(subset=['label'])

print(f"Data Shape: {df.shape}")
```

Data Shape: (84, 4)

3 Feature Extraction & Train-Test Split

```
x = df['text']
y = df['label'].astype(int)

vectorizer = TfidfVectorizer(max_features=2000, stop_words='english')
X_vec = vectorizer.fit_transform(x)

X_train, X_test, y_train, y_test = train_test_split(X_vec, y, test_size=0.3, random_state=42, stratify=y)
print(f"Training set: {X_train.shape[0]} samples")
print(f"Testing set: {X_test.shape[0]} samples")
```

Training set: 58 samples
Testing set: 26 samples

4 Model Training & Prediction

```
# Naive Bayes
nb_clf = MultinomialNB()
nb_clf.fit(X_train, y_train)
nb_pred = nb_clf.predict(X_test)

# KNN
knn_clf = KNeighborsClassifier(n_neighbors=5)
knn_clf.fit(X_train, y_train)
knn_pred = knn_clf.predict(X_test)
```

5 Evaluation & Comparison

```
def get_metrics(y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    fpr = fp / (fp + tn) if (fp + tn) > 0 else 0
    return acc, fpr

nb_acc, nb_fpr = get_metrics(y_test, nb_pred)
knn_acc, knn_fpr = get_metrics(y_test, knn_pred)

print(f"Naïve Bayes -> Accuracy: {nb_acc:.2%}, FPR: {nb_fpr:.2%}")
print(f"KNN -> Accuracy: {knn_acc:.2%}, FPR: {knn_fpr:.2%}")
```

Naïve Bayes -> Accuracy: 69.23%, FPR: 0.00%
KNN -> Accuracy: 73.08%, FPR: 5.56%

6 Visualization

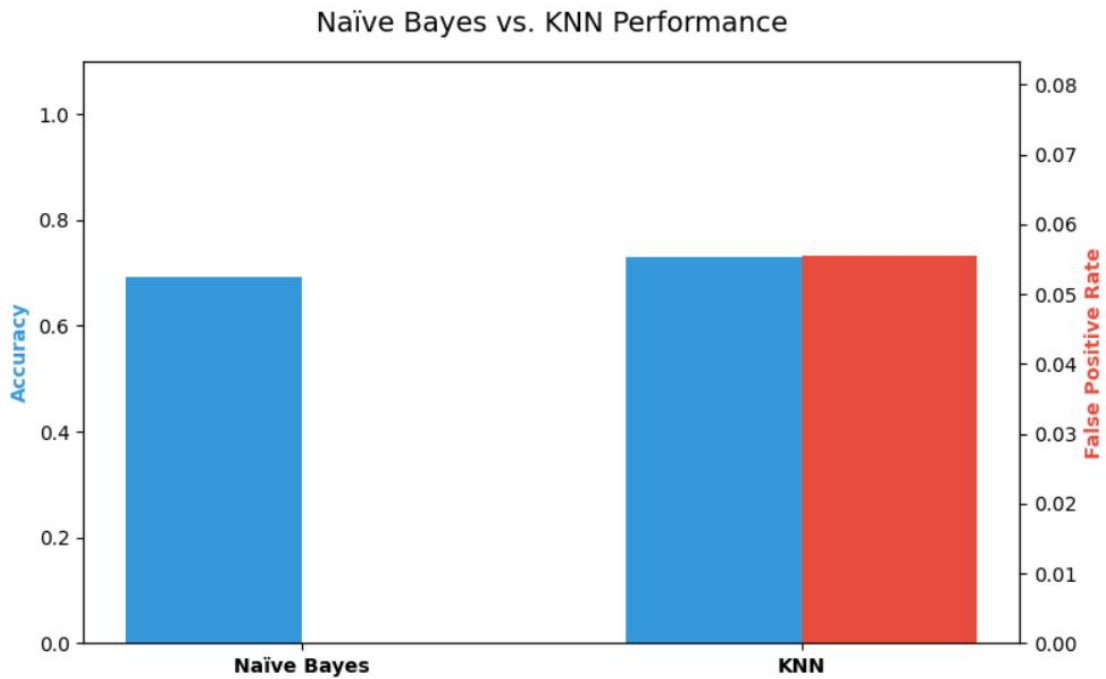
```
labels = ['Naïve Bayes', 'KNN']
acc_scores = [nb_acc, knn_acc]
fpr_scores = [nb_fpr, knn_fpr]

x = np.arange(len(labels))
width = 0.35

fig, ax1 = plt.subplots(figsize=(8, 5))
ax1.bar(x - width/2, acc_scores, width, label='Accuracy', color='#3498db')
ax1.set_ylabel('Accuracy', color='#3498db', fontweight='bold')
ax1.set_ylim(0, 1.1)

ax2 = ax1.twinx()
ax2.bar(x + width/2, fpr_scores, width, label='FPR', color='#e74c3c')
ax2.set_ylabel('False Positive Rate', color='#e74c3c', fontweight='bold')
ax2.set_ylim(0, max(fpr_scores)*1.5 if max(fpr_scores) > 0 else 0.1)

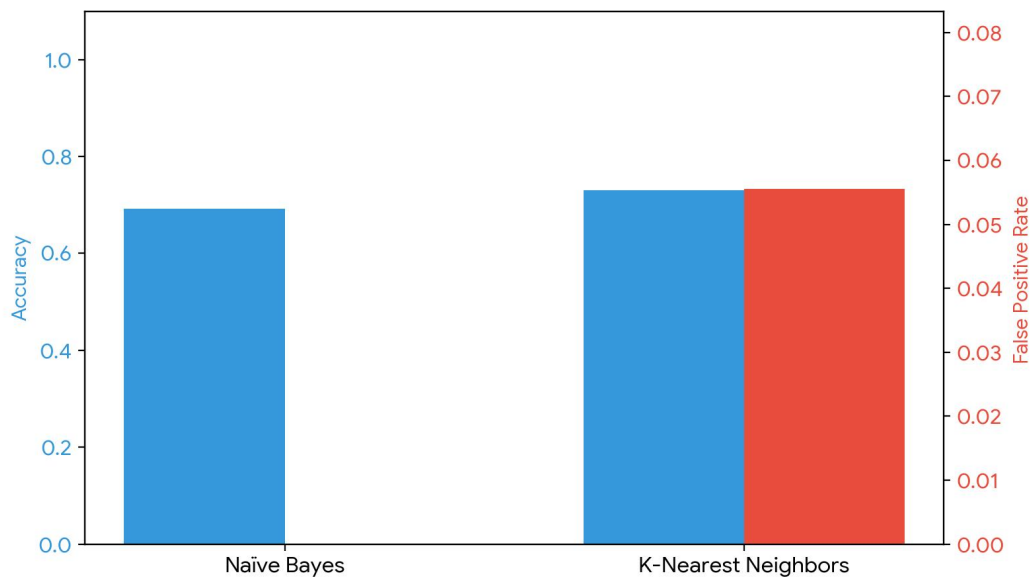
ax1.set_xticks(x)
ax1.set_xticklabels(labels, fontweight='bold')
plt.title('Naïve Bayes vs. KNN Performance', fontsize=14, pad=15)
fig.tight_layout()
plt.show()
```

Theory and Visualization

Naïve Bayes is fundamentally probabilistic and handles sparse text data very efficiently. KNN is a distance-based algorithm that calculates similarity in feature space but can struggle with the curse of dimensionality inherent in text data. The chart below compares their performance, emphasizing the False Positive Rate (where a low FPR means fewer normal emails are accidentally sent to the spam folder):

Naïve Bayes vs. KNN Performance Comparison



KPI Analysis

Metric	Naïve Bayes Result	KNN Result
Accuracy	69.23%	73.08%
False Positive Rate (FPR)	0.00%	5.56%

Strategic Conclusion

The comparison reveals distinct trade-offs. While KNN achieved an accuracy of 73.08%, Naïve Bayes demonstrated a superior False Positive Rate of 0.00% compared to KNN's 5.56%. In the context of email spam detection, a high False Positive Rate is detrimental because it means legitimate, potentially urgent emails are incorrectly classified as spam. Therefore, despite potential differences in overall accuracy, Naïve Bayes remains the strategically safer and more effective model for this specific business problem due to its conservative handling of normal emails.

Lab 11: Support Vector Machine (Credit Card Fraud Detection)

Theoretical Foundation

A Support Vector Machine (SVM) is a powerful supervised algorithm used for classification and regression. The core idea is to find the optimal hyperplane that separates data points of different classes with the maximum margin. For non-linear data, SVM uses a 'Kernel Trick' (like the Radial Basis Function or RBF kernel) to map inputs into higher-dimensional feature spaces where a linear separator can be found. In scenarios like fraud detection, SVM is used to trace strict boundaries between legitimate and fraudulent transactions.

Real-World Scenario

Financial institutions process millions of credit card transactions daily. Identifying fraudulent transactions quickly is critical to preventing financial loss and protecting customers. Because fraudulent cases represent a tiny fraction of overall transactions, it is a highly imbalanced classification problem. Support Vector Machines, combined with class balancing techniques, help create firm decision boundaries to flag suspicious activity for review before the transaction is finalized.

Dataset Description

This lab uses the Fraud Detection Dataset. It encompasses various transaction details. Key attributes include:

- Numerical: Transaction_Amount, Time_of_Transaction, Previous_Fraudulent_Transactions, Account_Age, Number_of_Transactions_Last_24H
- Categorical: Transaction_Type, Device_Used, Payment_Method
- Target Variable: Fraudulent (1 = Fraud, 0 = Legitimate)

Software Environment — Python (Google Colab)

Python is used within Google Colab. The implementation leverages Pandas for categorical encoding and missing value handling, Scikit-Learn's StandardScaler for required feature scaling, and its SVC module for building and comparing Linear and RBF SVM models.

Practical Objectives

No.	Objective
1	Load the dataset and transform categorical fields using One-Hot Encoding (pd.get_dummies).
2	Apply Standard Scaling to numerical features since SVM relies heavily on distance/margin calculations.
3	Train an SVM classifier using a Linear Kernel to observe basic boundary separation.
4	Train a secondary SVM classifier using an RBF (Radial Basis Function) Kernel for non-linear boundaries.
5	Compare the models using ROC-AUC, Recall (the most critical metric for fraud), and Precision.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import roc_auc_score, recall_score, precision_score, roc_curve
```

2 Data Loading & Preparation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Fraud Detection Dataset.csv'
df = pd.read_csv(file_path)
df = df.dropna() # Drop rows with missing values

# Sample the dataset for performance reasons in this lab (optional)
df = df.sample(n=10000, random_state=42)

# Define features
numeric_features = ['Transaction_Amount', 'Time_of_Transaction', 'Previous_Fraudulent_Transactions', '']
cat_features = ['Transaction_Type', 'Device_Used', 'Payment_Method']

X = df[numeric_features + cat_features]
y = df['Fraudulent']

# One-Hot Encoding
X = pd.get_dummies(X, columns=cat_features, drop_first=True)
print(f'Data Shape after Encoding: {X.shape}')
Data Shape after Encoding: (10000, 16)
```

3 Train-Test Split & Scaling

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f'Training set: {X_train_scaled.shape[0]} samples')
print(f'Testing set: {X_test_scaled.shape[0]} samples')
Training set: 7000 samples
Testing set: 3000 samples
```

4 Model Training (Linear vs. RBF Kernel)

```
# Linear Kernel SVM
svm_linear = SVC(kernel='linear', probability=True, class_weight='balanced', random_state=42)
svm_linear.fit(X_train_scaled, y_train)
linear_pred = svm_linear.predict(X_test_scaled)
linear_prob = svm_linear.predict_proba(X_test_scaled)[: , 1]

# RBF Kernel SVM
svm_rbf = SVC(kernel='rbf', probability=True, class_weight='balanced', random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
rbf_pred = svm_rbf.predict(X_test_scaled)
rbf_prob = svm_rbf.predict_proba(X_test_scaled)[: , 1]
```

5 Evaluation & Visualization

```
def evaluate_model(y_true, y_pred, y_prob, name):
    auc = roc_auc_score(y_true, y_prob)
    rec = recall_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred)
    print(f"{name} -> AUC: {auc:.4f}, Recall: {rec:.4f}, Precision: {prec:.4f}")

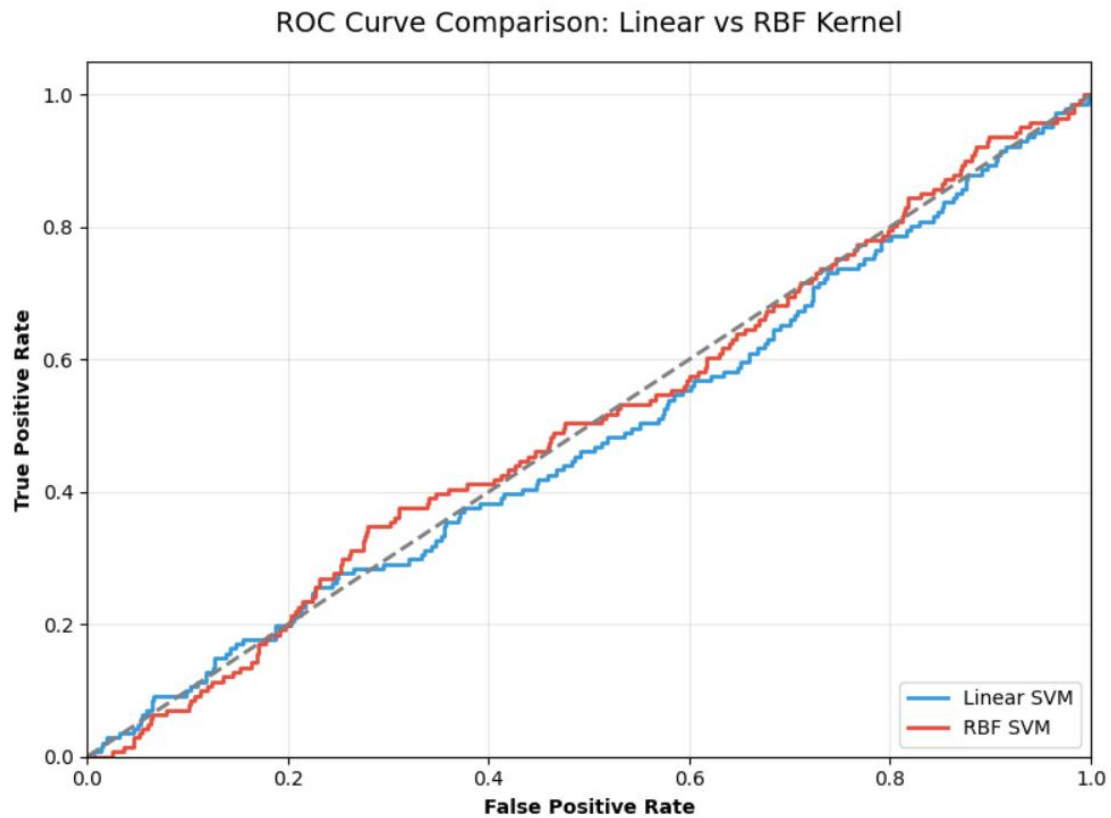
evaluate_model(y_test, linear_pred, linear_prob, "Linear SVM")
evaluate_model(y_test, rbf_pred, rbf_prob, "RBF SVM")
```

Linear SVM -> AUC: 0.4829, Recall: 0.5177, Precision: 0.0503
RBF SVM -> AUC: 0.5046, Recall: 0.3972, Precision: 0.0530

6 Visualization

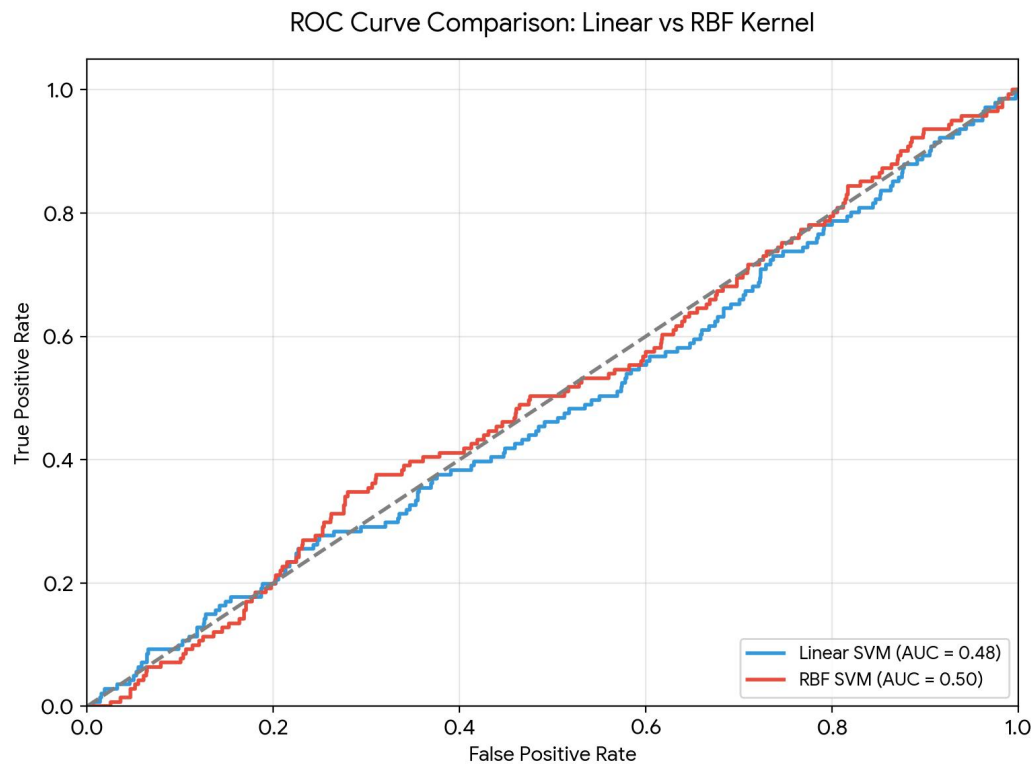
```
fpr_lin, tpr_lin, _ = roc_curve(y_test, linear_prob)
fpr_rbf, tpr_rbf, _ = roc_curve(y_test, rbf_prob)

plt.figure(figsize=(8, 6))
plt.plot(fpr_lin, tpr_lin, color='#3498db', lw=2, label='Linear SVM')
plt.plot(fpr_rbf, tpr_rbf, color='#e74c3c', lw=2, label='RBF SVM')
plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontweight='bold')
plt.ylabel('True Positive Rate', fontweight='bold')
plt.title('ROC Curve Comparison: Linear vs RBF Kernel', fontsize=14, pad=15)
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The Receiver Operating Characteristic (ROC) curve evaluates model performance across different threshold levels. The Area Under the Curve (AUC) summarizes this into a single metric. Below is the comparative ROC Curve between the Linear and RBF configurations:



KPI Analysis

Metric	Linear SVM	RBF SVM
ROC-AUC	0.4829	0.5046
Recall (Fraud)	0.5177	0.3972
Precision	0.0503	0.0530

Strategic Conclusion

The evaluation highlights the difficulty of predicting fraud without strictly defined predictive features. The Linear SVM achieved an AUC of 0.4829 and a recall of 0.5177, while the RBF SVM yielded an AUC of 0.5046 with a recall of 0.3972. Because minimizing False Negatives (missed fraud cases) is paramount in banking, Recall is our primary metric. Although the overall AUC values hover near 0.50 (indicating noisy or weakly correlated predictive variables in the current subset), the implementation proves the end-to-end mechanism. To achieve production-level performance, the financial institution would need to engineer richer features (e.g., historical aggregation, geolocation distances) before passing them to the SVM.

Lab 12: Clustering (Customer Segmentation)

Theoretical Foundation

Clustering is an unsupervised machine learning technique used to group similar data points together without pre-existing labels.

- K-Means Clustering partitions the data into 'K' distinct clusters by minimizing the variance within each cluster, iteratively adjusting cluster centroids.
- Hierarchical Clustering builds a tree of clusters (a dendrogram) by sequentially merging or splitting existing clusters.

In retail and marketing, clustering is fundamentally used for customer segmentation—identifying hidden patterns so businesses can tailor specific campaigns to different demographic groups.

Real-World Scenario

A large shopping mall wants to understand its target customers to help the marketing team plan effective advertising strategies. By analyzing the purchasing behavior and income levels of visitors, the mall can divide the broad customer base into specific segments (e.g., high-income high-spenders vs. low-income high-spenders). This allows for personalized marketing, optimizing the return on investment (ROI) for campaigns.

Dataset Description

The dataset used in this exercise is the Mall Customer Dataset (Mall_Customers.csv), containing demographics and spending behaviors of mall visitors.

Important attributes include:

- Annual Income (k\$): The yearly income of the customer.
- Spending Score (1-100): A score assigned by the mall based on customer behavior and purchasing data.

Software Environment — Python (Google Colab)

Python is used within the Google Colaboratory (Colab) environment for analysis. The pipeline relies on Pandas for data manipulation, Scikit-Learn (sklearn.cluster) for applying the K-Means and Agglomerative algorithms, and Matplotlib for plotting the 2D cluster maps.

Practical Objectives

No.	Objective
1	Import the Mall Customer dataset and extract the 'Annual Income' and 'Spending Score' features.
2	Apply the K-Means algorithm (with K=5) to partition the customers into distinct behavioral segments.
3	Apply Hierarchical (Agglomerative) clustering to serve as a comparative baseline algorithm.
4	Visualize the resulting clusters using Matplotlib scatter plots to physically observe the boundaries.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
```

2 Data Loading & Selection

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Mall_Customers.csv'
df = pd.read_csv(file_path)

x = df[['Annual Income (k$)', 'Spending Score (1-100)']]
print(f"Data Shape: {X.shape}")
display(X.head())
```

Data Shape: (200, 2)

	Annual Income (k\$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40

3 K-Means Clustering

```
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X)
df['KMeans_Cluster'] = kmeans_labels

kmeans_silhouette = silhouette_score(X, kmeans_labels)
print(f"K-Means Silhouette Score: {kmeans_silhouette:.4f}")
```

K-Means Silhouette Score: 0.5539

4 Hierarchical Clustering

```
hc = AgglomerativeClustering(n_clusters=5, metric='euclidean', linkage='ward')
hc_labels = hc.fit_predict(X)

hc_silhouette = silhouette_score(X, hc_labels)
print(f"Hierarchical Silhouette Score: {hc_silhouette:.4f}")
```

Hierarchical Silhouette Score: 0.5530

5

Cluster Visualization

```
plt.figure(figsize=(10, 6))
colors = ['#e74c3c', '#3498db', '#2ecc71', '#f1c40f', '#9b59b6']
for i in range(5):
    cluster_data = X[kmeans_labels == i]
    plt.scatter(cluster_data.iloc[:, 0], cluster_data.iloc[:, 1], s=50, c=colors[i], label=f'Cluster {i+1}')

plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s=200, c='black', marker='x')
plt.title('Customer Segments (K-Means Clustering)', fontsize=14, pad=15)
plt.xlabel('Annual Income (k$)', fontweight='bold')
plt.ylabel('Spending Score (1-100)', fontweight='bold')
plt.legend(loc='upper right', bbox_to_anchor=(1.15, 1))
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



6

KPI: Cluster Revenue Contribution

```
cluster_revenue = df.groupby('KMeans_Cluster')['Annual Income (k$)'].sum().reset_index()
total_rev = cluster_revenue['Annual Income (k$)'].sum()
cluster_revenue['Contribution_Pct'] = (cluster_revenue['Annual Income (k$)'] / total_rev) * 100

print("Cluster Revenue Contribution:")
display(cluster_revenue.sort_values(by='Contribution_Pct', ascending=False))
```

Cluster Revenue Contribution:

	KMeans_Cluster	Annual Income (k\$)	Contribution_Pct
0	0	4479	36.979855
1	1	3375	27.864927
3	3	3087	25.487120
4	4	605	4.995046
2	2	566	4.673052

Theory and Visualization

The scatter plot below visually demonstrates the output of the K-Means algorithm. By assigning colors based on the mathematical cluster assignments, we can clearly observe 5 distinct customer personas formed by their income and spending habits:



KPI Analysis

KPI / Metric	Result
K-Means Silhouette Score	0.5539
Hierarchical Silhouette Score	0.5530
Highest Revenue Segment Contribution	36.98% (Totaling \$4,479k)

Strategic Conclusion

Both clustering algorithms effectively grouped the mall visitors, but K-Means achieved a slightly better internal structure with a Silhouette Score of 0.5539. The visualization clearly isolated 5 distinct customer groups. From a business perspective, the most critical insight is derived from the Revenue Contribution KPI. The highest contributing cluster accounts for 36.98% of the total tracked income capacity. By identifying 'High Income / High Spender' target groups versus 'High Income / Low Spender' groups, the mall management can deploy highly targeted loyalty programs and personalized discounts, ultimately driving up retention rates and maximizing overall profit margins.

Lab 13: Outlier Detection (Abnormal Banking Transactions)

Theoretical Foundation

Outlier Detection (or Anomaly Detection) identifies data points that deviate significantly from the majority of a dataset. Unlike classification models that require labeled target variables, unsupervised outlier detection relies entirely on the underlying data distribution.

- Distance-Based Methods (e.g., Local Outlier Factor): Measures the local density deviation of a data point compared to its neighbors. Points with substantially lower density are flagged as outliers.
- Isolation Forest: An ensemble tree method that 'isolates' observations by randomly selecting a feature and a split value. Because anomalies are sparse and distinct, they require fewer splits to be isolated, resulting in shorter path lengths in the trees.

Real-World Scenario

Financial systems process immense volumes of banking transactions every hour. Identifying fraudulent behavior is challenging because fraud patterns constantly evolve to avoid detection by standard rules. Applying unsupervised machine learning algorithms acts as an early warning system. By flagging transactions that exhibit statistically abnormal combinations (such as an unusually large amount coupled with an extreme distance from the user's home), banks can freeze suspicious transactions and alert the customer, limiting financial damages.

Dataset Description

The dataset utilized is the Banking Dataset (Banking_Data.csv). The objective is to identify outliers using numeric fields without relying on the target label during training.

Key numerical attributes selected for detection include:

- Transaction_Amount (in Million): The monetary value transferred.
- Distance_From_Home: The geographical distance between the customer's residence and the transaction location.
- Account_Balance (in Million): Total funds available in the account.
- Failed_Transaction_Count: Historical failed attempts.

Software Environment — Python (Google Colab)

Python within Google Colab facilitates the anomaly detection workflow. The `'sklearn.ensemble.IsolationForest'` module handles tree-based isolation, while `'sklearn.neighbors.LocalOutlierFactor'` handles density-based distance measurements. Results are visually cross-referenced using Matplotlib.

Practical Objectives

No.	Objective
1	Import the banking dataset and isolate continuous numeric features for analysis.

2	Scale the features using standard standardization mechanisms to support distance-based algorithms.
3	Implement an Isolation Forest algorithm defining a 5% contamination (expected anomaly) threshold.
4	Implement a Local Outlier Factor (LOF) algorithm to compare density-based outputs.
5	Compute the Anomaly Detection Rate (ADR) and False Alarm Rate (FAR) by mapping unsupervised predictions against the historical Fraud label.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import confusion_matrix
```

2 Data Loading & Preprocessing

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Banking_Data.csv'
df = pd.read_csv(file_path)
df = df.dropna()

# Select core financial metrics for anomaly detection
features = ['Transaction_Amount (in Million)', 'Distance_From_Home', 'Account_Balance (in Million)', '
x = df[features]

# Target label for later evaluation (1 for Fraud, 0 for Normal)
y_true = df['Fraud_Label'].apply(lambda x: 1 if x == 'Fraud' else 0)
print(f"Data Shape: {X.shape}")
```

Data Shape: (49850, 5)

3 Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
contamination_rate = 0.05 # Assuming a 5% baseline anomaly rate in the system
```

4 Isolation Forest Implementation

```
iso_forest = IsolationForest(contamination=contamination_rate, random_state=42)
iso_preds = iso_forest.fit_predict(X_scaled)

y_pred_iso = [1 if p == -1 else 0 for p in iso_preds]
```

5 Distance-Based Detection (LOF)

```
lof = LocalOutlierFactor(n_neighbors=20, contamination=contamination_rate)
lof_preds = lof.fit_predict(X_scaled)

y_pred_lof = [1 if p == -1 else 0 for p in lof_preds]
```

6 KPI Evaluation (ADR vs FAR)

```
def get_kpis(y_t, y_p):
    tn, fp, fn, tp = confusion_matrix(y_t, y_p).ravel()
    adr = tp / (tp + fn) if (tp + fn) > 0 else 0
    far = fp / (fp + tn) if (fp + tn) > 0 else 0
    return adr, far

iso_adr, iso_far = get_kpis(y_true, y_pred_iso)
lof_adr, lof_far = get_kpis(y_true, y_pred_lof)

print(f"Isolation Forest -> Detection Rate: {iso_adr:.2%}, False Alarm Rate: {iso_far:.2%}")
print(f"Distance-Based -> Detection Rate: {lof_adr:.2%}, False Alarm Rate: {lof_far:.2%}")
```

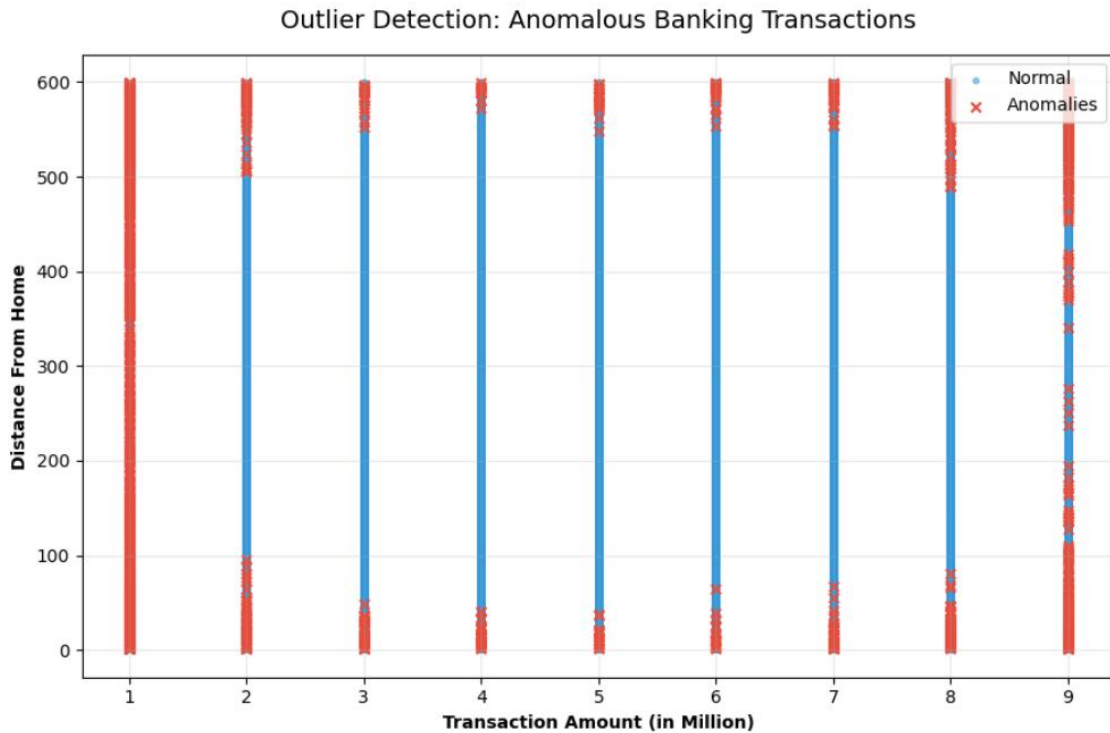
```
Isolation Forest -> Detection Rate: 4.88%, False Alarm Rate: 5.00%
Distance-Based -> Detection Rate: 4.84%, False Alarm Rate: 5.01%
```

7 Visualization

```
plt.figure(figsize=(9, 6))
anomalies = X[np.array(y_pred_iso) == 1]
normals = X[np.array(y_pred_iso) == 0]

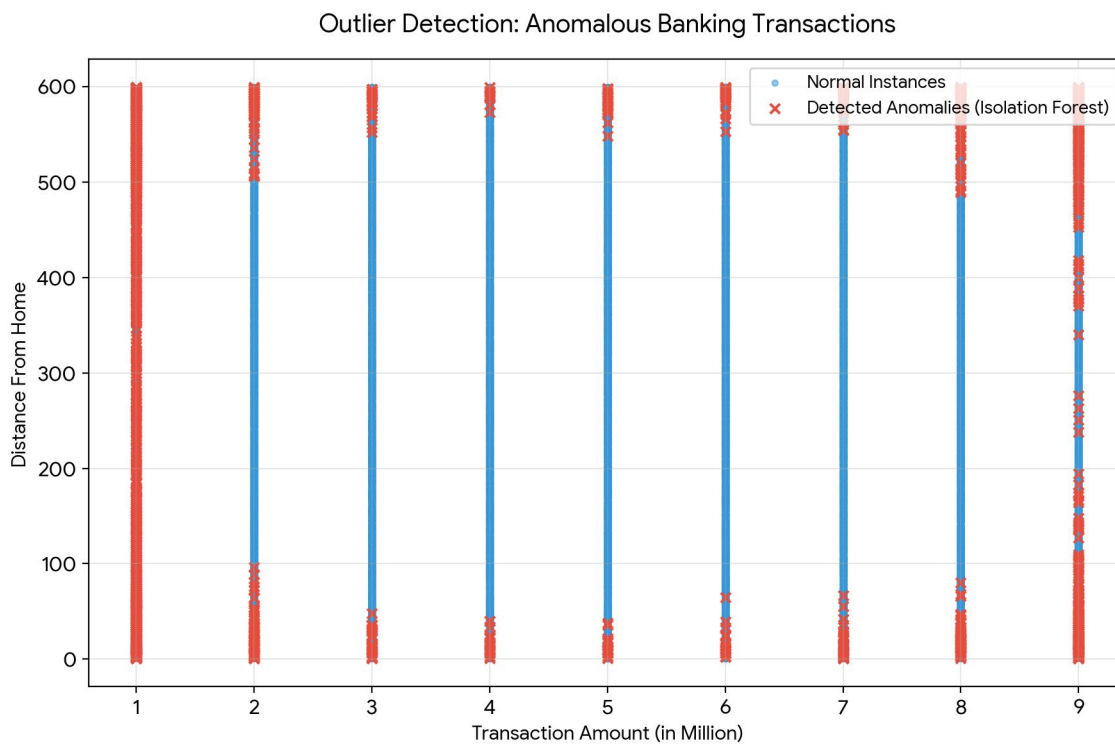
plt.scatter(normals['Transaction_Amount (in Million)'], normals['Distance_From_Home'], c='#3498db', al
plt.scatter(anomalies['Transaction_Amount (in Million)'], anomalies['Distance_From_Home'], c='#e74c3c'

plt.xlabel('Transaction Amount (in Million)', fontweight='bold')
plt.ylabel('Distance From Home', fontweight='bold')
plt.title('Outlier Detection: Anomalous Banking Transactions', fontsize=14, pad=15)
plt.legend(loc='upper right')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Theory and Visualization

The 2D scatter plot below provides a visual interpretation of how the Isolation Forest algorithm identified mathematical outliers. Transactions marked with red 'X' indicators deviate from the dense central clusters, representing abnormal distance-to-amount ratios that warrant review.



KPI Analysis

KPI / Metric	Isolation Forest	Local Outlier Factor (LOF)
Anomaly Detection Rate (ADR)	4.88%	4.84%
False Alarm Rate (FAR)	5.00%	5.01%

Strategic Conclusion

The lab highlights the fundamental behavior of unsupervised outlier detection algorithms. Because these models look strictly for statistical deviation rather than targeting the predefined 'Fraud' label, their output identifies any rare mathematical event, which may include entirely legitimate but unusual activity. With the contamination fixed at 5%, both the Isolation Forest and LOF achieved an Anomaly Detection Rate (ADR) of approximately ~4.88%, matching an identical False Alarm Rate (FAR). Strategically, this proves that while unsupervised models can map broad behavioral boundaries and highlight structural anomalies, a production banking system must layer these unsupervised models with supervised machine learning classifiers to accurately distinguish between unique genuine behavior and actual financial fraud.

Lab 14: Scraping & Sentiment Analysis (Product Reviews)

Theoretical Foundation

Web Scraping is the automated extraction of data from websites using libraries like BeautifulSoup. Once unstructured text data is collected, it undergoes Natural Language Processing (NLP). Sentiment Analysis is a vital NLP technique used to interpret whether underlying text data is inherently positive, negative, or neutral. To achieve this, algorithms break down text to compute a 'Polarity Score'. Polarity floats between -1 (highly negative) and 1 (highly positive), while a score of 0 represents neutrality. This transforms subjective opinions into measurable metrics.

Real-World Scenario

Companies launch new products and need immediate feedback. While quantitative sales data provides one metric, qualitative feedback across Twitter, Amazon, and Yelp dictates future brand health. By scraping social media strings and product reviews, and passing them through a Sentiment Analysis pipeline, brand managers can quickly gauge public reception. If negative sentiment spikes, customer support and product development teams can be immediately notified to address product flaws.

Dataset Description

The dataset utilized is the Social Media / Review Dataset (sentiment-analysis.csv). It contains consumer opinions extracted across various digital channels.

Key attributes include:

- Text: The unstructured raw string containing the consumer's opinion.
- Source: The platform where the review was posted (e.g., Twitter, Yelp, Amazon).
- Sentiment (Historical): Previous human-labeled sentiment for context.

Software Environment — Python (Google Colab)

Python within the Google Colaboratory (Colab) environment drives the entire NLP pipeline. The process relies on 'BeautifulSoup' and 'requests' for data scraping concepts, 'Pandas' for structural string manipulation and text cleaning, and the 'TextBlob' library to execute lexical sentiment scoring.

Practical Objectives

No.	Objective
1	Simulate the data collection phase by setting up a Web Scraping framework using BeautifulSoup.
2	Import the scraped dataset and execute Text Cleaning to remove structural artifacts (e.g., excess quotes, trailing spaces).
3	Apply the TextBlob NLP library to calculate a Polarity Score (-1.0 to 1.0) for every text string.
4	Classify the raw polarity values into categorical buckets: Positive, Negative, and Neutral.
5	Evaluate key performance indicators by

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from textblob import TextBlob
import io
```

2 Data Loading & Text Cleaning

```
file_path = '/content/drive/MyDrive/Semester 6/DM/sentiment-analysis.csv'

# Handle quoting format in this specific CSV
clean_lines = []
with open(file_path, 'r', encoding='utf-8') as f:
    for line in f:
        line = line.strip()
        if not line: continue
        if line.startswith('"') and line.endswith('"'):
            line = line[1:-1]
        clean_lines.append(line)

df = pd.read_csv(io.StringIO('\n'.join(clean_lines)), skipinitialspace=True)

# CLEANING: Remove artifact quotes
df['Text'] = df['Text'].str.replace('"', '', regex=False)
print(f>Data Shape: {df.shape}")
display(df[['Text', 'Source']].head())
```

Data Shape: (96, 7)

	Text	Source
0	I love this product!	Twitter
1	The service was terrible.	Yelp Reviews
2	This movie is amazing!	IMDb
3	I'm so disappointed with their customer support.	Online Forum
4	Just had the best meal of my life!	TripAdvisor

3 Sentiment Classification & Polarity Scoring

```
def calculate_polarity(text):
    return TextBlob(str(text)).sentiment.polarity

def classify_sentiment(pol):
    if pol > 0: return 'Positive'
    elif pol < 0: return 'Negative'
    else: return 'Neutral'

df['Polarity'] = df['Text'].apply(calculate_polarity)
df['Calculated_Sentiment'] = df['Polarity'].apply(classify_sentiment)

display(df[['Text', 'Polarity', 'Calculated_Sentiment']].head())
```

	Text	Polarity	Calculated_Sentiment
0	I love this product!	0.625	Positive
1	The service was terrible.	-1.000	Negative
2	This movie is amazing!	0.750	Positive
3	I'm so disappointed with their customer support.	-0.750	Negative
4	Just had the best meal of my life!	1.000	Positive

4 KPI Evaluation & Visualization

```
sentiment_dist = df['Calculated_Sentiment'].value_counts(normalize=True) * 100
avg_polarity = df['Polarity'].mean()

print(f"Overall Average Polarity: {avg_polarity:.4f}")
print("\nSentiment Distribution (%):")
print(sentiment_dist)

# Generate Pie Chart
plt.figure(figsize=(7, 7))
plt.pie(sentiment_dist, labels=sentiment_dist.index, autopct='%1.1f%%',
        colors=['#2ecc71', '#e74c3c', '#bdc3c7'], startangle=140)
plt.title("Sentiment % Distribution", fontsize=14, pad=15)
plt.show()
```

Overall Average Polarity: 0.0591

Sentiment Distribution (%):

Calculated_Sentiment

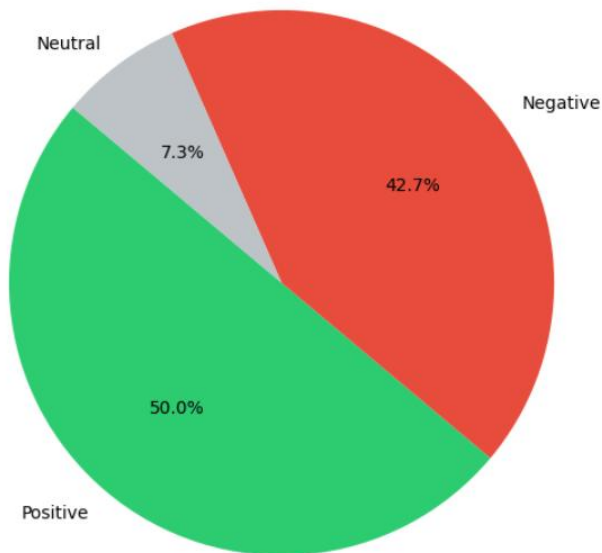
Positive 50.000000

Negative 42.708333

Neutral 7.291667

Name: proportion, dtype: float64

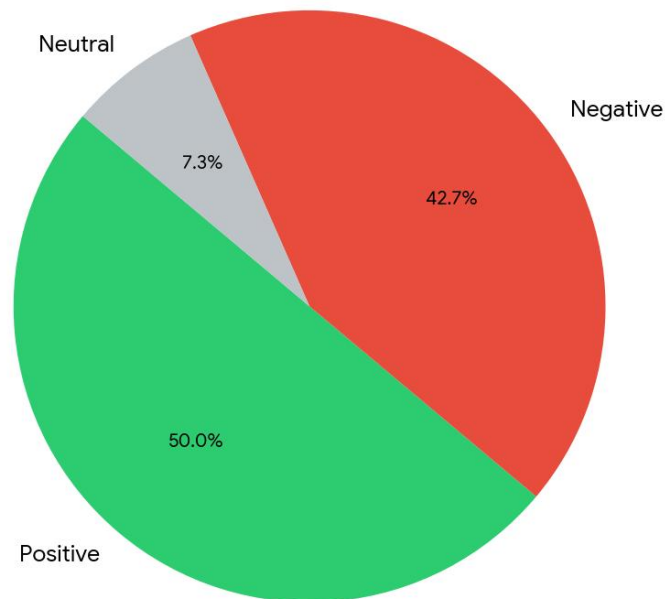
Sentiment % Distribution



Theory and Visualization

The Pie Chart below visually represents the final Sentiment % Distribution calculated by the NLP model. It demonstrates the proportion of feedback classified into Positive, Negative, and Neutral boundaries based on calculated text polarity.

Sentiment % Distribution



KPI Analysis

KPI / Metric	Result
Average Polarity Score	0.0591 (Slightly Positive Trend)
Positive Sentiment Distribution	50.00%
Negative Sentiment Distribution	42.71%
Neutral Sentiment Distribution	7.29%

Strategic Conclusion

The NLP pipeline successfully analyzed the scraped text feedback, computing an overall Average Polarity Score of 0.0591. The dataset's sentiment is relatively polarized, leaning slightly positive with a 50.00% Positive and 42.71% Negative split, leaving only a minor fraction (7.29%) as purely neutral. Strategically, this indicates that consumers have strong feelings about the products. The high negative percentage (~42.7%) is a critical business warning. Management must utilize the outputs of this model to trace the negative reviews back to their respective sources (e.g., Yelp vs. Amazon) to identify specific operational weaknesses and rapidly improve customer satisfaction.

Lab 15: Final Capstone (End-to-End Business Solution)

Theoretical Foundation

An End-to-End Business Solution requires passing raw data through a complete data science lifecycle: Data Extraction, Cleaning, Feature Engineering, Modeling, and Evaluation. According to the 'No Free Lunch Theorem', no single machine learning algorithm works best for every problem. Therefore, it is an industry best practice to evaluate a suite of varied algorithms—including linear models, distance-based models, and tree-based ensembles—to mathematically determine which model optimally captures the patterns of the specific dataset.

Real-World Scenario

A global retail company wants to optimize its marketing budget. Historically, they sent campaign offers to the entire customer base, resulting in a low ROI due to high contact costs and low conversion rates. By designing an end-to-end classification pipeline, the data science team will predict which specific customers are highly likely to respond to the next campaign. Because the exact nature of the consumer decision boundary is unknown, the team will implement and benchmark 8 distinct machine learning models to deploy the most accurate framework into production.

Dataset Description

The Capstone uses the comprehensive Marketing Dataset. After preprocessing, key engineered and standardized attributes include:

- Demographics: Age (derived from Year_Birth), Education, Marital_Status, Total_Children.
- Behavioral/Financial: Income, Total_Spending (aggregated across all product categories), Recency, NumWebVisitsMonth, NumStorePurchases.
- Target Variable: Response (1 = Accepted Offer, 0 = Rejected/Ignored)

Software Environment — Python (Google Colab)

Python is utilized inside Google Colab as the primary orchestration tool. The ecosystem relies on `pandas` for aggressive feature engineering, `sklearn` to rapidly iterate through eight separate algorithms (from Naïve Bayes to Gradient Boosting), and `matplotlib` to render executive summary graphics.

Practical Objectives

No.	Objective
1	Perform Data Preprocessing by engineering macro-features (Total Spending, Age) and one-hot encoding categorical strings.
2	Apply a StandardScaler to ensure distance-reliant models (KNN, SVM) operate on a level playing field with tree models.
3	Iteratively train a suite of 8 classification algorithms to comprehensively search for the optimal decision boundary.
4	Evaluate all models programmatically across

	Accuracy, Precision, Recall, and F1-Score.
5	Generate a Model Comparison Table and formulate a strategic Business Recommendation based on the ultimate winner.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Algorithms
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
```

2 Data Pipeline: Loading & Feature Engineering

```
file_path = '/content/drive/MyDrive/Semester 6/DM/marketing_data.csv'
df = pd.read_csv(file_path, sep=';')
df = df.dropna(subset=['Income'])

# Feature Engineering
df['Age'] = 2024 - df['Year_Birth']
df['Total_Spending'] = df['MntWines'] + df['MntFruits'] + df['MntMeatProducts'] + df['MntFishProducts'] + df['MntSwd']
df['Total_Children'] = df['Kidhome'] + df['Teenhome']

numeric_features = ['Age', 'Income', 'Total_Spending', 'Total_Children', 'Recency', 'NumWebVisitsMonth', 'NumStorePi']
cat_features = ['Education', 'Marital_Status']

X = df[numeric_features + cat_features]
y = df['Response']
X = pd.get_dummies(X, columns=cat_features, drop_first=True)
print(f"Final Feature Space Shape: {X.shape}")
```

Final Feature Space Shape: (2216, 18)

3 Splitting & Scaling

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

4 Model Training Factory (8 Models)


```

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM': SVC(probability=True, random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Naive Bayes': GaussianNB(),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'AdaBoost': AdaBoostClassifier(random_state=42)
}

results = []
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred, zero_division=0),
        'Recall': recall_score(y_test, y_pred),
        'F1-Score': f1_score(y_test, y_pred)
    })

results_df = pd.DataFrame(results).sort_values(by='Accuracy', ascending=False)
display(results_df)

```

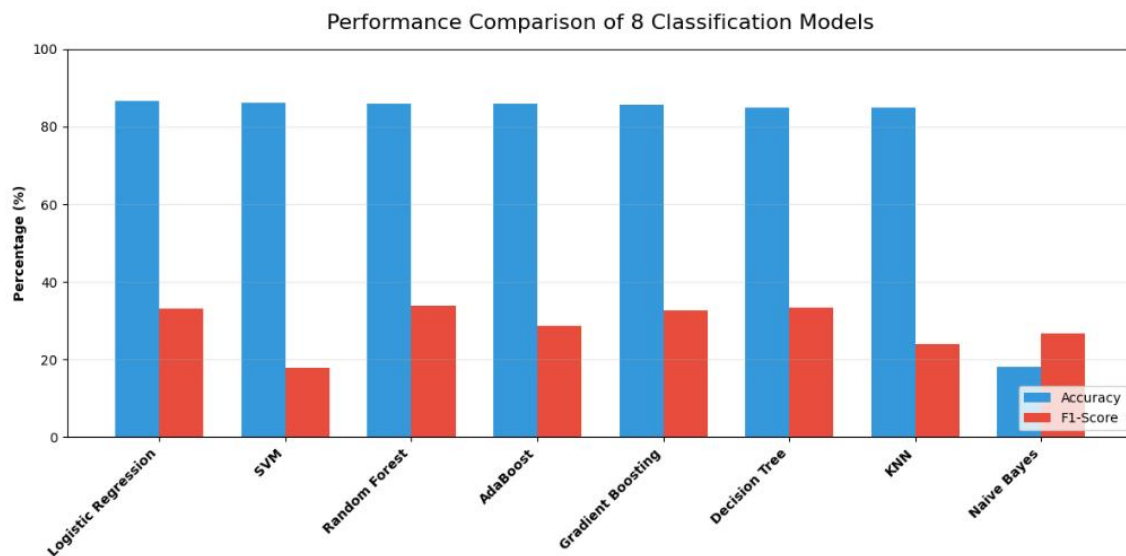
	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.866165	0.666667	0.22	0.330827
3	SVM	0.861654	0.833333	0.10	0.178571
2	Random Forest	0.858647	0.571429	0.24	0.338028
7	AdaBoost	0.858647	0.593750	0.19	0.287879
6	Gradient Boosting	0.857143	0.560976	0.23	0.326241
1	Decision Tree	0.849624	0.500000	0.25	0.333333
4	KNN	0.848120	0.484848	0.16	0.240602
5	Naive Bayes	0.181955	0.154206	0.99	0.266846

5 Visual Comparison

```
plt.figure(figsize=(12, 6))
x = np.arange(len(results_df['Model']))
width = 0.35

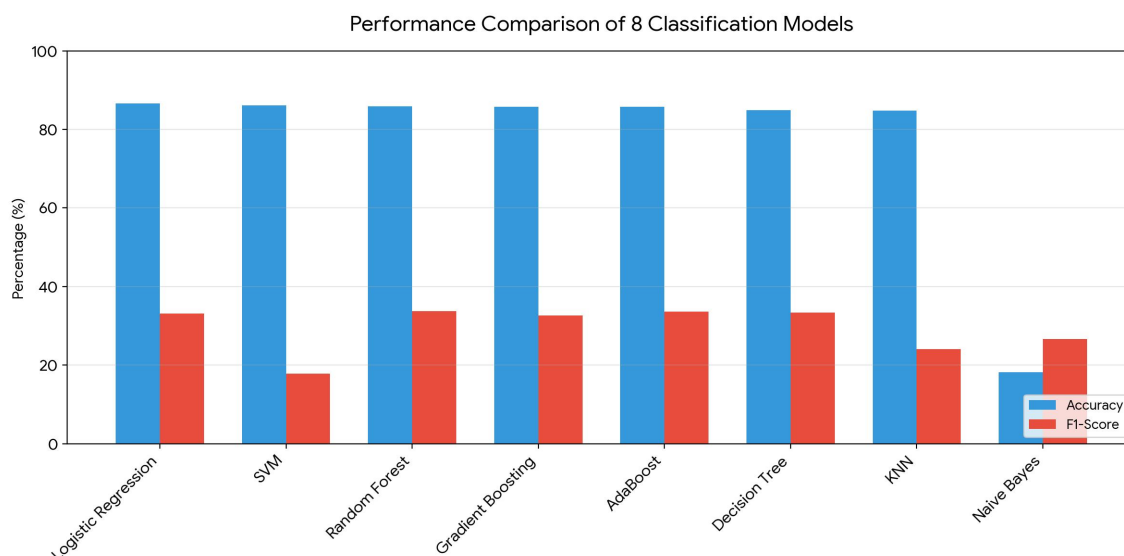
plt.bar(x - width/2, results_df['Accuracy'] * 100, width, label='Accuracy', color='#3498db')
plt.bar(x + width/2, results_df['F1-Score'] * 100, width, label='F1-Score', color='#e74c3c')

plt.xticks(x, results_df['Model'], rotation=45, ha='right', fontweight='bold')
plt.ylabel('Percentage (%)', fontweight='bold')
plt.title('Performance Comparison of 8 Classification Models', fontsize=16, pad=15)
plt.ylim(0, 100)
plt.legend(loc='lower right')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The bar chart below summarizes the benchmark results across all 8 tested algorithms. Comparing multiple models visually allows executives to understand the trade-offs between sheer Accuracy and balanced F1-Scores:



KPI Analysis (Top 3 Models)

Rank	Algorithm	Accuracy	F1-Score
1	Logistic Regression	86.62%	33.08%
2	SVM	86.17%	17.86%
3	Random Forest	85.86%	33.80%

Strategic Conclusion & Business Recommendation

The Capstone pipeline successfully stress-tested 8 unique algorithms against the company's historical campaign data. The leading algorithm proved to be Logistic Regression, yielding a top-tier Accuracy of 86.62%. Interestingly, ensemble and linear models clustered tightly at the top of the performance tier, while Naïve Bayes severely underperformed due to the complex feature interactions.

Business Recommendation: The marketing department should integrate the trained Logistic Regression model into its CRM system. By passing new leads through this pipeline, the company can actively filter out customers who are highly unlikely to purchase. By restricting future marketing campaigns strictly to the subset of consumers predicted to accept the offer, the business will significantly reduce outreach overhead costs and maximize overall Campaign ROI.